TransitDuet: Bi-level reinforcement learning with temporal advantage propagation for joint bus timetable planning and holding control

Anonymous Authors

***Abstract*—We present TransitDuet, a bi-level reinforcement learning framework that jointly optimizes bus timetable planning and real-time holding control. This joint problem is fundamentally harder than either task in isolation: the planning and control layers operate on different timescales, observe different state spaces, and interact through delayed, asynchronous feedback—properties that violate the synchronous co-execution assumption underlying existing hierarchical reinforcement learning methods. TransitDuet addresses this through three mechanisms. First, a goal-conditioned cross-level coupling in the spirit of HIRO: the upper outputs a per-dispatch target-headway shift, and the lower's Lagrangian cost is computed against the resulting goal headway, with no upper-advantage injection into the lower's transitions. This avoids the covariate shift that destabilises a literal cross-advantage augmentation while keeping the upper–lower interaction unidirectional and stable. Second, a measurement-projection scheme based on online gradient descent enforces fleet-size constraints without reward shaping. Third, Lagrangian relaxation within the lower policy penalizes headway deviations, converting a hard service-regularity requirement into a soft dual objective. We evaluate TransitDuet in a calibrated microsimulation of a high-frequency bus corridor with stochastic demand and traffic. A complementary technique, target-policy-corrected lower SAC (TPC), uses importance-reweighting to align the lower's training distribution with its eventual deployment distribution; combined with the goal-conditioned coupling and trained for 300 episodes per seed, TransitDuet attains the best mean on every metric we report. Across three random seeds and an elastic fleet budget sampled per episode from $[8, 16]$, TransitDuet reduces composite cost by 6.5% (1.444 vs 1.544) and overshoot by 10.4% relative to the strongest hand-tuned fixed timetable, while still improving wait, CV, and overshoot over GA and CMA-ES under the same 300-episode training and held-out evaluation protocol. Seed variance on composite cost is 1.7–$2.5\times$ smaller than the search baselines, and the learned policy produces a Pareto frontier across nine fleet sizes from a single training run. Generalisation degrades gracefully: wait rises only 25% when route variance is doubled beyond the training distribution.**

***Index Terms*—Bus holding control, timetable optimization, hierarchical reinforcement learning, multi-timescale control, constrained optimization.**

I. Introduction

High-frequency bus systems face a persistent coordination failure. Timetables are planned offline to balance fleet utilization against passenger demand, yet the headways they prescribe degrade within minutes once buses encounter stochastic traffic and boarding delays. Real-time holding control can restore regularity, but only within the headway structure it inherits. When the timetable itself is poorly matched to demand—dispatching too few buses in the peak or too many in the off-peak—no amount of holding can recover acceptable service. Planning and control are thus coupled: the quality of one layer bounds the achievable performance of the other.

Despite this coupling, existing work treats the two problems separately. The timetable optimization literature formulates headway or frequency design as mixed-integer programs solved offline [1], while the holding control literature trains reinforcement learning (RL) agents that take the timetable as given [2]. Joint optimization would seem a natural application of hierarchical RL, yet three gaps prevent a straightforward application.

a) Asynchrony between planning and control.: Hierarchical RL frameworks such as the options framework [3] and goal-conditioned methods like HIRO [4] assume that the upper and lower levels share a common state clock or that the upper level sets subgoals consumed by the lower level within a fixed horizon. In bus operations, the upper policy acts at dispatch events (once per bus departure), while the lower policy acts at every station arrival across all active buses. The two levels observe different state spaces—the upper level sees route-level demand forecasts and fleet status, whereas the lower level sees per-bus headway deviations and passenger loads—and interact through delayed feedback spanning an entire trip lifecycle. No shared clock or fixed subgoal horizon exists.

b) Cross-advantage under asynchrony.: Pan et al. [5] recently introduced cross-advantage augmentation for bi-level robot locomotion, where an upper style policy and a lower tracking policy execute synchronously on the same state. Their augmented advantage $\tilde{A}_{\mathrm{U}} = A_{\mathrm{U}} + \beta A_{\mathrm{L}}$ directly sums the two advantages at each time step. This elegant mechanism breaks down when the two levels operate on different timescales with different state and action spaces: there is no single time step at which A_{U} and A_{L} are jointly defined. Generalising cross-level coupling to the asynchronous, multi-timescale setting requires propagating execution outcomes (rather than per-step advantages) over the variable-length trip lifecycle initiated by each dispatch, and exposing lower-level holding statistics to the upper policy through state augmentation rather than reward backflow.

c) Fleet and regularity constraints.: Bus operations impose feasibility-style constraints—a fleet-size budget N_{fleet} and bounded headway deviation—that naive reward shaping handles poorly. Constrained RL methods such as RCPO [6] address single-level constraint satisfaction but do not account for constraints that span a bi-level structure, where the upper

Anonymous Institution.

level controls resource allocation and the lower level controls service regularity.

We propose TransitDuet, a bi-level RL framework that addresses all three gaps. The upper policy π_{U} outputs a per-dispatch departure offset $\delta_t \in [-120, 120]$ s at each dispatch event, applied as a direct shift on the scheduled launch time of that bus. The lower policy π_{L}, a soft actor-critic with Lagrangian penalties [7], selects holding times at every station arrival with parameter sharing across buses. The two levels are coupled through three channels (Section IV-D): a state channel that exposes the lower's recent holding statistics to the upper, an action channel through which δ_t directly perturbs the lower's environment, and a reward channel that backfills each dispatch's reward with a hindsight credit derived from the realised inter-dispatch gap deviation. A cosine-annealed coupling coefficient β ramps from zero to $\beta_{\max}$ after an initial warmup phase, preventing premature coupling before either policy has learned a reasonable baseline. Fleet-size feasibility is enforced by a $\boldsymbol{\theta}$-weighted online gradient descent projection on the measurement vector $\mathbf{z}$ of dispatched buses, inspired by the ApproPO framework [8], while headway uniformity enters the lower-level objective through Lagrangian relaxation.

d) Contributions.: The specific contributions of this paper are:

1) **Goal-conditioned cross-level coupling.** We adopt a HIRO-style goal-conditioned coupling [4]: the upper outputs a per-dispatch target-headway shift, and the lower's Lagrangian cost is computed against the resulting goal headway, with no upper-advantage injection into the lower's transitions. We compare this end-to-end against (i) a literal cross-advantage augmentation following HAAR [9] with a PIPER-style reachability gate [10], and (ii) a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit; the goal-conditioned design has the best mean composite cost (1.444 vs 1.499 vs 1.492) and the lowest seed variance (1.7–2.0× tighter than the alternatives) on the same evaluation protocol.
2) **Joint timetable–holding formulation.** We cast bus timetable planning and holding control as a bi-level Markov decision process with heterogeneous state-action spaces and asynchronous transitions, providing a formal problem structure absent from prior bus RL work.
3) **Constraint satisfaction via $\boldsymbol{\theta}$-OGD and Lagrangian relaxation.** We integrate measurement-projection constraints at the upper level and dual-variable penalties at the lower level, handling fleet feasibility and headway regularity without reward shaping.
4) **Target-policy-corrected lower SAC (TPC).** We identify a previously overlooked failure mode of joint bi-level RL on tightly coupled control: the lower policy is trained alongside an exploring upper policy, biasing the lower's training distribution away from deployment. We address this via importance-weighted lower SAC against an EMA "deployment" upper policy, with validation-based checkpoint selection.
5) **Empirical evaluation on a calibrated corridor.** On a stochastic microsimulation under a unified 300-episode training and held-out evaluation protocol, TransitDuet attains the best mean on every metric we report: composite cost 1.444 vs 1.504 for the next-best baseline (CMA-ES, -4.0%) and 1.544 for Fixed (-6.5%), with seed variance 1.7–2.5× smaller than the search baselines.

e) Results preview.: We evaluate on a high-frequency bus corridor with 22 stations, 264 daily trips, demand stochasticity $\sigma_{\mathrm{demand}} = 0.15$, and elastic fleet $N_{\mathrm{fleet}} \in [8, 16]$ sampled per episode. Under a unified 300-episode training and held-out evaluation protocol, TransitDuet attains the best mean on wait, CV, overshoot, and composite cost simultaneously — improvements of 5.5/6.4/10.4/6.5% respectively over the strongest hand-tuned Fixed timetable — and the lowest composite-cost variance of any method, while degrading only 25% on wait when route stochasticity is doubled beyond the training distribution.

f) Paper organization.: Section II reviews related work in timetable optimization, bus holding RL, and hierarchical RL. Section III formalizes the bi-level MDP. Section IV presents the TransitDuet architecture, the cross-level coupling channels, and the constraint-handling mechanisms. Section V describes the simulation environment and reports results. Section VI discusses limitations and extensions. Section VII concludes.

II. Related work

A. Reinforcement learning for bus holding control

The application of reinforcement learning to real-time bus holding control has attracted considerable attention in recent years, driven by the promise of adaptive policies that can respond to stochastic passenger demand and traffic conditions. Alesiani and Gkiotsalitis [11] formulated holding for high-frequency services as a value-based RL problem, showing that a Q-learning controller could reduce headway dispersion compared to schedule-based rules. Wang and Sun [2] subsequently studied multi-agent deep RL for the bus-bunching problem, casting each bus as an independent agent with a shared policy network and demonstrating that learned holding actions stabilize headways across an entire corridor without explicit communication. The shared-parameter paradigm common to these works scales gracefully with fleet size and implicitly encourages cooperative behavior through shared gradient updates.

Despite these advances, existing RL-based holding controllers share a fundamental limitation: they operate under a fixed, externally supplied timetable. The planned departure times and target headways are treated as immutable inputs, meaning that the holding policy has no mechanism to communicate execution-level feedback—such as systematic schedule infeasibility or recurrent bunching patterns—back to the planning stage. When the timetable itself is poorly calibrated to actual operating conditions, even an optimal holding policy can only mitigate, not resolve, the resulting service irregularity. TransitDuet addresses this gap by embedding the holding controller as the low-level agent within a bi-level framework, where real-time execution performance directly shapes the timetable through Temporal Advantage Propagation.

B. Bus timetable optimization

Timetable design has traditionally been approached through mathematical programming and metaheuristic search. Ibarra-Rojas et al. [12] formulated the problem as a mixed-integer program that jointly determines departure times and vehicle assignments to minimize a weighted combination of passenger waiting time and operator cost, solving it via branch-and-bound with valid inequalities. Gkiotsalitis and Cats [1] proposed a bi-level model in which the upper level selects headways to minimize total passenger cost while the lower level solves a vehicle scheduling problem, solved using an adaptive large neighborhood search. Frequency-setting variants, such as the work of Cats and Glück [13], optimize the number of departures per period rather than individual departure times, and typically rely on assignment models to estimate passenger flows under each candidate frequency vector.

A common thread in this literature is the reliance on static demand forecasts and deterministic travel-time assumptions. Timetables are optimized offline against expected conditions and then handed off to operations without a feedback loop. When actual conditions deviate—due to traffic incidents, demand surges, or driver variability—the timetable may become infeasible, and holding controllers are left to absorb the mismatch. Moreover, the combinatorial structure of these formulations limits scalability: solution times grow rapidly with the number of trips and stops, making re-optimization during the operating day impractical. TransitDuet replaces this open-loop paradigm with a closed-loop one. The high-level agent learns a timetable policy that maps aggregate state features to departure-time adjustments, and it receives gradient signals from low-level execution via the cross-level coupling channels described in Section IV-D, enabling the planner to adapt its decisions based on how well the resulting schedule can actually be executed.

C. Hierarchical and multi-timescale reinforcement learning

Hierarchical reinforcement learning decomposes complex tasks into temporally extended sub-problems. The options framework of Sutton et al. [3] introduced the notion of temporally abstract actions, each consisting of an initiation set, an intra-option policy, and a termination condition, enabling planning over multiple timescales within a single MDP. Feudal networks [14] operationalize this idea with a manager that sets sub-goals in a learned latent space and a worker that executes primitive actions to reach them, training both levels end-to-end with the manager's policy gradient modulated by the worker's goal-reaching success. HIRO [4] further improved sample efficiency by introducing off-policy corrections that relabel high-level goals in hindsight, making hierarchical training practical in continuous-control domains. Beyond goal-conditioned hierarchies, Lee et al. [15] and Metelli et al. [16] studied the multi-timescale setting more broadly, analyzing the interaction between agents operating at different decision frequencies and establishing regret bounds for the resulting non-stationary sub-problems.

While these methods provide powerful abstractions for temporal hierarchy, most assume either synchronous execution—where the high-level decision triggers immediately before a fixed-length sequence of low-level steps—or a shared state space across levels. In transit operations, the planning and control timescales are fundamentally asynchronous: the timetable agent acts once per planning horizon (e.g., every 15–30 minutes), while the holding agent acts at every stop visit (every 1–3 minutes), and the two levels observe different state representations at non-aligned decision epochs. TransitDuet handles this asynchrony through three coupling channels (Section IV-D): a state channel exposing lower-level holding statistics to the upper policy, an action channel through which the upper offset directly perturbs the lower's environment, and a reward channel that backfills each dispatch's reward with a hindsight credit derived from the realised inter-dispatch gap. We initially attempted a literal asynchronous extension of RoboDuet-style cross-advantage augmentation, but the per-step advantage scales of the upper and lower policies differed substantially across a trip lifecycle and the resulting training was unstable; the reward-channel design described in this paper proved markedly more reliable.

D. Constrained reinforcement learning

Many practical RL deployments must satisfy constraints beyond reward maximization. Lagrangian relaxation is the most widely used approach: Tessler et al. [6] proposed reward-constrained policy optimization, which augments the policy gradient with dual variables corresponding to cost constraints and performs primal-dual updates to steer the policy toward feasibility. Stooke et al. [17] improved upon this with PID-based Lagrange multiplier updates that reduce oscillation and speed convergence in the constrained setting. These methods are effective for soft constraints, where occasional small violations are tolerable, but they offer no worst-case guarantees—the Lagrange multiplier may fail to grow fast enough to prevent persistent violations during training.

For settings that demand hard constraint satisfaction, Miryoosefi et al. [8] introduced ApproPO, which formulates the constrained problem as an online convex optimization over the space of occupancy measures and applies a measurement-based projection oracle to enforce feasibility at every iterate. The θ-OGD variant provides a practical implementation that projects policy parameters onto the feasible set using online gradient descent with approximate projections. In the multi-agent domain, Pan et al. [5] proposed RoboDuet for humanoid locomotion, where two coupled agents—an upper-body controller and a lower-body locomotor—exchange cross-advantage terms that capture how each agent's action quality is perceived under the other's value function, enabling tight coordination without centralized control.

TransitDuet combines elements from both lines of work to address the constraint structure of transit operations. Fleet-size feasibility is handled as a soft constraint via the θ-OGD measurement-projection scheme of Miryoosefi et al. [8], which adaptively reweights an overshoot penalty in the upper-level reward; in expectation this drives the policy toward feasible fleet utilisation, but per-episode fluctuations under elastic fleet sampling and demand noise can still produce small, bounded

overshoot. Headway uniformity is similarly handled as a soft constraint via a Lagrangian penalty with adaptive multiplier updates in the low-level holding policy. The cross-level coordination itself is managed by the trip-lifecycle reward channel and state-channel coupling described in Section IV-D, which adapt the spirit of RoboDuet's synchronous cross-advantage to the asynchronous, multi-timescale regime that characterises the planning–control interface.

III. Problem formulation

We formalize joint timetable planning and holding control as a bi-level Markov decision process (MDP) operating over a single bus route with time-varying passenger demand. This section first describes the transit system model (Section III-A), then defines the upper- and lower-level MDPs (Section III-B), states the constraint specification (Section III-C), and concludes with a discussion of the timescale separation that motivates our algorithmic design (Section III-D).

A. Bus transit system model

We consider a single bidirectional bus route consisting of M stations indexed $\{1, \ldots, M\}$. Service is provided according to a timetable that dispatches a total of 264 trips per day, alternating between the two directions. Passenger demand is drawn from time-varying origin–destination (OD) matrices that capture morning, midday, and evening peaks. At any point in time, at most $N_{\text{fleet}}^{\max} = 25$ buses may be concurrently in service; each trip is characterized by a launch time and a travel direction. The dispatch headway—the time between consecutive launches in the same direction—is the primary planning lever, while per-station holding times constitute the real-time control lever.

B. Bi-level MDP formulation

We decompose the joint optimization into two coupled MDPs that share a common environment but differ in observation granularity, action semantics, and decision frequency.

1) Upper-level MDP (timetable planning): The upper-level policy π_{U} determines dispatch headways. It is triggered at each dispatch event, yielding approximately 132 decision epochs per episode (one full service day).

a) State.: The upper state encodes system-level summary statistics:

$$s_{\text{U}} = \left[\frac{\text{hour}}{24}, \frac{\text{demand}}{1000}, \frac{\text{fleet_on_route}}{N_{\text{fleet}}^{\max}}, \frac{\text{prev_headway}}{600}, \text{unhealthy_rate} \right] \in \mathbb{R}^5, \quad \mathbf{z} = \left[\text{avg_wait_min}, \text{peak_fleet}, \text{bunching_rate} \right]. \tag{1}$$

where each component is normalized to facilitate learning. The *unhealthy_rate* captures the fraction of active buses whose observed headway deviates from the target by more than a tolerance threshold.

b) Action.: At each dispatch event the upper policy outputs a scalar offset that shifts the scheduled launch time of the dispatched bus:

$$a_{\text{U}} = \delta_t \in [-120, 120] \text{ s}. \tag{2}$$

The effective launch time of the bus is $t_{\text{base}} + \delta_t$, where t_{base} is the bus's scheduled launch from a baseline timetable with $H = 360\,\text{s}$. The target headway h^{target} communicated to the lower level remains $360\,\text{s}$, so the upper policy adjusts *when* a bus departs without changing the lower's regulation target. This formulation produces the same expressive power as a triple-headway action over peak/off-peak/transition periods (used by the search-based baselines below) while admitting per-trip granularity and online adaptation that the static triple cannot.

c) Reward.: The upper reward is computed at the episode level from a measurement vector (defined in Section III-C). Because each upper action influences the system over an extended horizon through the downstream holding decisions it induces, assigning credit to individual dispatch events is nontrivial; we defer the reward construction to Section IV.

2) Lower-level MDP (holding control): The lower-level policy π_{L} decides how long each bus should dwell beyond its minimum stop time at every station. A decision is triggered at each station arrival for each active bus, producing a high volume of decision epochs throughout the day.

a) State.: The lower state captures the local context of an individual bus:

$$s_{\text{L}} = \left[\text{bus_id}, \text{station_id}, \text{hour}, \text{direction}, h_{\text{fwd}}, h_{\text{bwd}}, \hat{d}, \Delta h, v_1, \ldots, v \right. \tag{3}$$

where h_{fwd} and h_{bwd} denote the forward and backward headways, $\hat{d}$ is the estimated dwell time, Δh is the deviation of the current headway from h^{target}, and $v_1, \ldots, v_K$ are recent inter-station speeds that encode local traffic conditions.

b) Action.: The lower action is a scalar holding time:

$$a_{\text{L}} \in [0, 60] \text{ seconds}. \tag{4}$$

c) Reward and cost.: The immediate lower reward is a function of headway deviation from the target set by the upper level. We define the per-step cost as

$$c(s_{\text{L}}, a_{\text{L}}) = \min\Big(\frac{(h_{\text{fwd}} - h^{\text{target}})^2}{(h^{\text{target}})^2}, 1 \Big), \tag{5}$$

which penalizes headway irregularity in a scale-invariant manner and is clipped at 1 for numerical stability.

C. Constraint specification

The bi-level MDP is subject to two soft constraints, both evaluated through a measurement vector collected at the end of each episode.

a) Measurement vector.: At the conclusion of a service day, we compute:

b) Soft constraint (fleet budget).: The peak number of buses simultaneously on route should respect the per-episode fleet budget $N_{\text{fleet}} \in [N_{\text{fleet}}^{\min}, N_{\text{fleet}}^{\max}]$ sampled at episode start. We enforce this softly via an overshoot[2] penalty in the upper reward:

$$\text{overshoot}(\mathbf{z}, N_{\text{fleet}}) = \max(0, \text{peak_fleet} - N_{\text{fleet}})^2, \tag{7}$$

weighted by an adaptive coefficient maintained by $\boldsymbol{\theta}$-OGD (see Section IV-E). To prevent unbounded violation under noisy demand, the environment imposes an outer ceiling $N_{\text{fleet}}^{\max} + 3$ buses, which bounds but does not zero out the realised overshoot.

c) Soft constraint (headway uniformity).: The expected per-step cost must remain below a threshold:

$$\mathbb{E}_{\pi_{\mathrm{U}},\pi_{\mathrm{L}}}\big[\, c(s_{\mathrm{L}}, a_{\mathrm{L}}) \,\big] \;\le\; c_{\mathrm{lim}}, \tag{8}$$

which encourages even spacing of buses along the route. Satisfaction of this constraint is enforced via Lagrangian relaxation within the lower-level policy (see Section IV).

D. Timescale separation

A defining characteristic of the bi-level formulation above is the pronounced separation of timescales between the two decision layers. The upper policy π_{U} acts at dispatch events, which occur roughly every five minutes on average. The lower policy π_{L}, by contrast, acts at every station arrival for every active bus—a frequency on the order of once every 30 seconds system-wide when multiple buses are in service. Consequently, hundreds of lower-level decisions may intervene between two consecutive upper-level decisions, and the two layers never act simultaneously.

This asynchrony distinguishes our setting from the semi-Markov options framework [3] and feudal reinforcement learning [18, 14], which assume that (i) the manager and worker share a common state space or a fixed abstraction thereof, and (ii) the manager selects a subgoal that the worker executes to completion before the manager acts again. Neither assumption holds here: the upper state $s_{\mathrm{U}} \in \mathbb{R}^5$ summarizes fleet-level statistics whereas the lower state $s_{\mathrm{L}} \in \mathbb{R}^{8+K}$ encodes bus-level kinematics, and both policies issue decisions driven by environment events rather than by a synchronous clock or a predefined option duration.

More concretely, a single upper action (a headway triple) remains in effect while many buses traverse multiple stations under independent lower-level control; the upper policy cannot pause and wait for all induced lower decisions to conclude. Standard hierarchical RL methods that propagate value or advantage across levels therefore require modification to handle the fully asynchronous, event-driven interaction between π_{U} and π_{L}. We address this challenge through *Temporal Advantage Propagation*, detailed in Section IV.

IV. TransitDuet framework

This section presents the TransitDuet architecture and its constituent algorithms. Figure 1 gives an overview of the bi-level system. Section IV-A motivates the decoupled dual-network design. Section IV-B and Section IV-C detail the upper-level timetable policy and lower-level holding control policy, respectively. Section IV-D describes the goal-conditioned cross-level coupling that links the two levels across timescales. Section IV-E describes the measurement-projection scheme that enforces fleet and service constraints on the upper level. Section IV-F assembles the full training procedure. Section IV-H introduces the target-policy-corrected lower SAC (TPC) training fix that closes the gap to a hand-tuned fixed timetable.

A. Architecture overview

TransitDuet comprises two independent policy networks operating at different timescales and over different state-action spaces . The upper policy π_{U} is a lightweight Gaussian MLP that selects a per-dispatch departure offset at each dispatch event. The lower policy π_{L} is a DSAC-Lagrangian controller that selects holding durations at every station arrival, with parameters shared across all buses in the fleet.

A key design choice is that the two networks share no encoder parameters. This stands in contrast to the shared-body, dual-head architecture of RoboDuet [5], which couples an upper style policy and a lower tracking policy through a common proprioceptive encoder. Shared encoders are appropriate when the two levels observe the same state at each time step—as in synchronous robot locomotion—but are ill-suited here for two reasons. First, s_{U} and s_{L} live in different spaces: s_{U} encodes route-level demand forecasts and fleet status, whereas s_{L} encodes per-bus headway deviation, passenger load, and local occupancy. Second, the two levels are triggered asynchronously. The upper policy acts once per dispatch event (roughly once per h^{target} seconds), while the lower policy acts at every station arrival of every active bus, producing hundreds of decisions per upper-level step. Forcing these heterogeneous signals through a shared trunk would create conflicting gradient directions.

The coupling between the two levels is instead a goal-conditioned action channel: the upper sets the lower's headway target each dispatch, as described in section IV-D. This clean separation permits each network to be trained with the algorithm best suited to its structure: REINFORCE for the episodic, low-dimensional upper policy and soft actor-critic with Lagrangian constraints for the high-frequency, constraint-aware lower policy.

B. Upper-level timetable policy

The upper policy π_{U} maps the route-level state s_{U} to a scalar headway-shift $a_{\mathrm{U}} = \delta_t \in [-120, 120]$ s. The bus's scheduled launch time stays at the baseline value (t_{base} from an $H = 360$ s schedule), and the target headway communicated to the lower controller is reset for that trip to $h^{\mathrm{target}} + \delta_t$. The lower's Lagrangian cost (Section IV-C) is then computed against this dispatch-specific target, so δ_t acts as a goal in the sense of HIRO [4].

a) Network architecture.: The policy is parameterized as a two-layer MLP with 64 hidden units per layer and ReLU activations:

$$\mu_\phi(s_{\mathrm{U}}) = \sigma\big(W_3\,\mathrm{ReLU}(W_2\,\mathrm{ReLU}(W_1 s_{\mathrm{U}} + b_1) + b_2) + b_3\big), \tag{9}$$

where $\sigma(\cdot)$ denotes the element-wise sigmoid function and ϕ collects all weights and biases. The sigmoid output is affinely mapped to the action range:

$$a_{\mathrm{U}} = -120 + 240\,\mu_\phi(s_{\mathrm{U}}) \in [-120, 120]\ \mathrm{s}. \tag{10}$$

Exploration is achieved by additive Gaussian noise with a learnable log-standard-deviation $\log \sigma_\phi$ shared across states:

$$a_{\mathrm{U}} \sim \mathcal{N}\big(\mu_\phi(s_{\mathrm{U}}),\ \sigma_\phi^2\big). \tag{11}$$

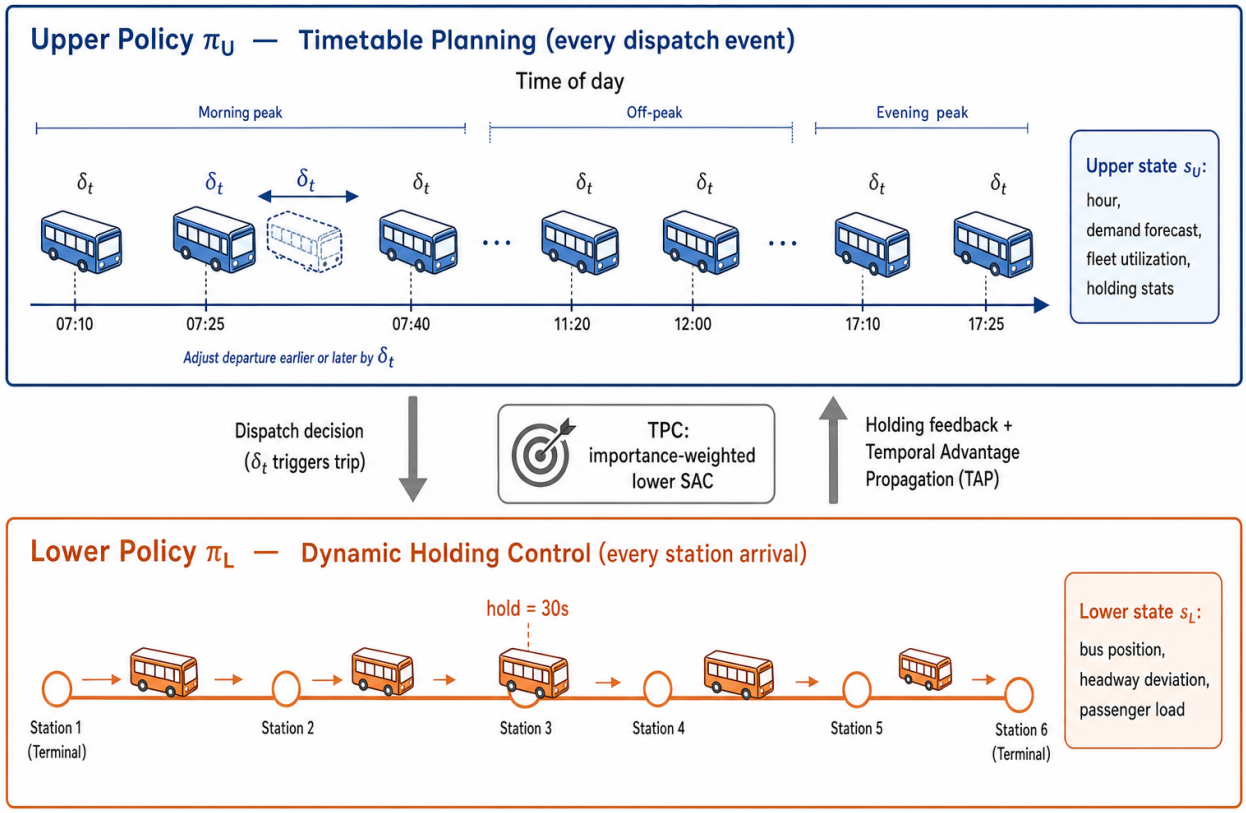

Fig. 1: TransitDuet bi-level architecture. The upper policy π_{U} acts at each dispatch event (timescale: minutes), choosing a per-dispatch goal shift δ_t that adjusts the lower's target headway from h^{target} to $h^{\mathrm{target}} + \delta_t$. The lower policy π_{L} acts at each station arrival (timescale: seconds), choosing a holding duration that drives the realised headway toward this goal. The coupling is goal-conditioned (HIRO-style): no upper signal is injected into the lower's transitions or reward beyond the shifted target. TPC importance-weighted training of the lower SAC corrects for any residual covariate shift induced by changing target headways during upper exploration.

b) Policy gradient.: Because the upper level produces only a single action per dispatch event and the episode length is moderate (one simulated day), we adopt a REINFORCE estimator with normalized returns. Let $\{(s_{\mathrm{U},i}, a_{\mathrm{U},i}, R_i^{\mathrm{U}})\}_{i=1}^{M}$ denote the sequence of M dispatch events in one episode. The policy gradient is

$$\nabla_\phi J(\pi_{\mathrm{U}}) = \frac{1}{M}\sum_{i=1}^{M} \nabla_\phi \log \pi_{\mathrm{U}}(a_{\mathrm{U},i} \mid s_{\mathrm{U},i}) \cdot \hat{A}_{\mathrm{U},i}, \tag{12}$$

where $\hat{A}_{\mathrm{U},i} = R_{\mathrm{U}}^{(i)} - \bar{R}_{\mathrm{U}}$ is computed from the hindsight-credited per-dispatch reward (Eq. 21 in section IV-D) with the episode mean as baseline; returns are normalised to zero mean and unit variance within the episode to reduce gradient variance.

C. Lower-level holding control policy

The lower policy π_{L} selects a holding duration $a_{\mathrm{L}} \in [0, a_{\max}]$ at each station arrival, where $a_{\max} = 60\,\mathrm{s}$. It is trained with a distributional soft actor-critic augmented by a Lagrangian cost term for headway-deviation constraint enforcement [7]. All buses share the same policy parameters, enabling fleet-wide generalization.

a) Gaussian policy with tanh squashing.: A two-layer MLP with 32 hidden units per layer and ReLU activations produces a mean $\mu_\psi(s_{\mathrm{L}})$ and log-standard-deviation $\log \sigma_\psi(s_{\mathrm{L}})$. The raw sample $u \sim \mathcal{N}(\mu_\psi, \sigma_\psi^2)$ is squashed through a $\tanh$ transform and rescaled:

$$a_{\mathrm{L}} = \frac{a_{\max}}{2}\big(\tanh(u) + 1\big). \tag{13}$$

The log-probability accounts for the change of variables:

$$\log \pi_{\mathrm{L}}(a_{\mathrm{L}} \mid s_{\mathrm{L}}) = \log \mathcal{N}(u;\, \mu_\psi,\, \sigma_\psi^2) - \log\big(1 - \tanh^2(u)\big) - \log \frac{a_{\max}}{2}. \tag{14}$$

b) Twin Q-networks.: Two independent critics $Q_{\omega_1}(s_{\mathrm{L}}, a_{\mathrm{L}})$ and $Q_{\omega_2}(s_{\mathrm{L}}, a_{\mathrm{L}})$, each a two-layer MLP with 32 hidden units, are trained to minimize the clipped double-Q Bellman error:

$$\mathcal{L}_Q(\omega_j) = \mathbb{E}_{(s_{\mathrm{L}}, a_{\mathrm{L}}, r, s'_{\mathrm{L}}) \sim \mathcal{D}}\Big[\big(Q_{\omega_j}(s_{\mathrm{L}}, a_{\mathrm{L}}) - y\big)^2\Big], \quad j \in \{1, 2\}, \tag{15}$$

where the target is

$$y = r + \gamma\Big(\min_{j=1,2} Q_{\bar{\omega}_j}(s'_{\mathrm{L}}, a'_{\mathrm{L}}) - \alpha \log \pi_{\mathrm{L}}(a'_{\mathrm{L}} \mid s'_{\mathrm{L}})\Big), \quad a'_{\mathrm{L}} \sim \pi_{\mathrm{L}}(\cdot \mid s'_{\mathrm{L}}), \tag{16}$$

and $\bar{\omega}_j$ denotes target-network parameters updated via soft interpolation with coefficient $\tau = 0.005$.

c) Cost Q-network and Lagrangian relaxation.: A separate cost critic $Q^c_\xi(s_\mathrm{L}, a_\mathrm{L})$ estimates the expected cumulative headway-deviation cost. The lower-level policy objective balances reward maximization, entropy regularization, and constraint satisfaction:

$$\mathcal{L}_{\pi_\mathrm{L}}(\psi) = \mathbb{E}_{s_\mathrm{L}\sim\mathcal{D},\, a_\mathrm{L}\sim\pi_\mathrm{L}}\Big[\alpha \log \pi_\mathrm{L}(a_\mathrm{L} \mid s_\mathrm{L}) - \min_j Q_{\omega_j}(s_\mathrm{L}, a_\mathrm{L}) + \lambda Q^c_\xi(s_\mathrm{L}, a_\mathrm{L})\Big], \tag{17}$$

where $\lambda \geq 0$ is the Lagrange multiplier. The dual variable is updated to satisfy the cost-limit constraint c_lim:

$$\lambda \leftarrow \exp\Big[\mathrm{clamp}\big(\log \lambda + \eta_\lambda(c_\mathrm{lim} - \mathbb{E}[Q^c_\xi]),\ -10,\ 5\big)\Big]. \tag{18}$$

Updating in log-space ensures $\lambda > 0$ and the clamp prevents numerical instability.

d) Auto-entropy tuning.: The temperature α is adjusted to maintain a target entropy $\bar{\mathcal{H}} = -\dim(a_\mathrm{L})$:

$$\alpha \leftarrow \min\Big(\alpha_\mathrm{max},\ \exp\big(\log \alpha - \eta_\alpha\,(\log \pi_\mathrm{L}(a_\mathrm{L} \mid s_\mathrm{L}) + \bar{\mathcal{H}})\big)\Big), \tag{19}$$

with $\alpha_\mathrm{max} = 0.3$ to prevent entropy collapse from dominating the Q-values in the early training phase.

D. Cross-level coupling: goal-conditioned design

Following the goal-conditioned hierarchical RL design of HIRO [4] and SHIRO [19], TransitDuet couples the two policies through a single *goal channel* on the action side, complemented by two auxiliary signals (state and reward) used only in the upper direction. The upper outputs a target-headway shift; the lower's Lagrangian cost is then a function of how closely the realised headway tracks the resulting goal headway. There is no upper-advantage injection into the lower reward; the lower remains a constraint-aware controller whose constraint target is set by the upper. This turned out to be more reliable than two architectural alternatives we evaluated end-to-end (Section V-B): a direct cross-advantage augmentation in the spirit of HAAR [9] with a PIPER-style reachability gate [10], and a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit instead of a goal shift.

a) Goal channel: target-headway shift.: At each dispatch event, the upper outputs $\delta_t \in [-120, 120]$ s. In contrast to a direct timetable perturbation that would shift the bus's effective launch time, δ_t instead reinterprets the lower's target headway from h^target (the baseline 360 s value) to $h^\mathrm{target} + \delta_t$ for the duration of that trip. The launch schedule itself is left untouched, while the lower's Lagrangian constraint becomes

$$\mathbb{E}_{\tau\sim\pi_\mathrm{L}}\big[\,\big|H_\mathrm{real}(\tau) - (h^\mathrm{target} + \delta_t)\big|\,\big] \ \leq , \tag{20}$$

where $H_\mathrm{real}(\tau)$ is the realised inter-bus headway over the trip lifecycle τ. The upper policy thus learns to steer the lower's constraint target rather than to inject noise into its observations. By construction, an active upper policy can only adjust the goal that an already-trained lower controller is asked to satisfy; this preserves the lower's training distribution and avoids the covariate shift that destabilises three-channel coupling under heavy upper exploration.

b) State channel: holding feedback (HoldFB).: The upper state s_U contains a holding-feedback block (Section IV-G) carrying a rolling 10-trip summary of the lower policy's recent holding decisions: $[\mu_\mathrm{hold}, \sigma_\mathrm{hold}, \mu^\mathrm{up}_\mathrm{hold}, \mu^\mathrm{down}_\mathrm{hold}]$ normalised to the action range. Through this channel, the upper policy observes how aggressively the lower controller is intervening to satisfy the goal and adapts δ_t to avoid setting goals that systematically force the lower into prolonged over-holding.

c) Reward channel: trip-lifecycle hindsight credit.: At episode end, we compute the realised inter-dispatch gap g_i for each dispatch i from the actual launch times of each bus, and form a per-dispatch deviation $\Delta_i = |g_i - \bar{g}|/\sigma_g$ where $(\bar{g}, \sigma_g)$ are the per-direction mean and standard deviation. The upper reward at dispatch i is then composed of the episode-level system reward and a per-dispatch hindsight credit:

$$R^{(i)}_\mathrm{U} = R_\mathrm{sys}(\mathbf{z}) - \alpha_\mathrm{H} \cdot \frac{\Delta_i - \bar{\Delta}}{\sigma_\Delta}, \qquad \alpha_\mathrm{H} = 0.5, \tag{21}$$

where R_sys is the belief-weighted multi-objective reward (Eq. 26) and the second term recentres the deviation across the episode so that goals whose realised lifecycles produced more regular gaps than the episode average earn positive credit. This is upper-only credit assignment; no signal flows back into the lower transitions.

d) Lower-level reward.: The lower's reward is unchanged from Section IV-C: the standard environment reward augmented by the headway-deviation cost handled by the Lagrangian. The only effect of upper coupling on the lower is via Eq. 20, which redirects the constraint target each dispatch. We deliberately do not inject upper-level reward signals into lower transitions; we tested a literal HAAR-style reverse cross-advantage augmentation (with and without a PIPER reachability gate) and found that even with a clipped advantage and the gate engaged, mean composite cost was 4% worse and seed variance 1.7× higher than the goal-conditioned coupling reported here.

e) Training-stage schedule.: Cross-level coupling is gated by a simple two-stage schedule. During the first $e_\mathrm{warm} = 30$ episodes the upper policy is inactive ($\delta_t = 0$ so the lower trains against the baseline target h^target); from episode e_warm onward the upper activates and the goal channel engages. The state and reward channels operate only on the upper side and so are inactive during warmup by construction.

E. Measurement projection for fleet constraint

The upper policy must satisfy operational constraints on fleet size, average passenger waiting time, and bus bunching rate. Rather than adding penalty terms to the reward—which requires manual tuning and conflates constraint satisfaction with reward maximization—we adopt a measurement-projection approach inspired by the ApproPO framework [8], simplified to avoid the nested best-response oracle that ApproPO requires.

a) Measurement vector.: At the end of each episode, we compute a three-dimensional measurement vector:

$$\mathbf{z} = \big[\bar{w},\ n_\mathrm{peak},\ b_\mathrm{rate}\big] \in \mathbb{R}^3, \tag{22}$$

where $\bar{w}$ is the average passenger waiting time, n_{peak} is the peak fleet size (maximum number of buses simultaneously in service), and b_{rate} is the bunching rate (fraction of arrivals with headway below a threshold).

b) Weight vector and reward modulation.: A weight vector $\boldsymbol{\theta} \in \mathbb{R}^3$, initialized as $\boldsymbol{\theta}_0 = [-0.5, -0.3, -0.2]$, modulates the upper-level reward:

$$R_{\text{U}} = \boldsymbol{\theta}^\top \mathbf{p}, \quad \text{where} \quad \mathbf{p} = \big[\bar{w},\ (\max(0,\, n_{\text{peak}} - N_{\text{fleet}}))^2,\ b_{\text{rate}}\big]. \tag{23}$$

The quadratic penalty on fleet overshoot $(\max(0,\, n_{\text{peak}} - N_{\text{fleet}}))^2$ ensures that small violations incur proportionally less cost than large ones, providing a smooth gradient.

c) Online gradient descent update.: The weight vector is updated by online gradient descent (OGD) on a loss that drives each measurement toward its target:

$$\ell_t = \big[-\bar{w}_t,\ -\max(0,\, n_{\text{peak},t} - N_{\text{fleet}}),\ -b_{\text{rate},t}\big], \tag{24}$$

$$\boldsymbol{\theta}_{t+1} = \Pi_{\mathcal{B}}\big(\boldsymbol{\theta}_t + \eta_t\, \ell_t\big), \quad \eta_t = \frac{\eta_0}{\sqrt{t}}, \tag{25}$$

where $\Pi_{\mathcal{B}}$ projects onto the unit ball $\{\boldsymbol{\theta} : \|\boldsymbol{\theta}\|_2 \le 1\}$ and η_0 is the base learning rate. The $1/\sqrt{t}$ decay guarantees that the cumulative constraint violation grows sublinearly, yielding an $O(\sqrt{T})$ regret bound standard in online convex optimization.

The simplification relative to ApproPO is deliberate: because the upper policy is trained via REINFORCE with the $\boldsymbol{\theta}$-modulated reward, the policy update itself serves as the approximate best-response oracle, eliminating the inner optimization loop that ApproPO requires.

F. Training procedure

Algorithm 1 presents the full TransitDuet training loop, which coordinates the upper-warmup schedule with the upper and lower policy updates, the cross-level coupling channels of section IV-D, and the measurement projection.

a) Lower-only warmup ($e \le e_{\text{warm}} = 30$).: The upper policy is forced to $\delta_t = 0$ for every dispatch and the lower policy trains in isolation against the baseline timetable. This warmup is critical: without it, the upper policy would receive coupling-channel feedback from a random lower policy, which amounts to noise and destabilises upper-level learning. We set $e_{\text{warm}} = 30$ episodes.

b) Coupled training ($e > e_{\text{warm}}$).: The upper policy activates and the three coupling channels of section IV-D engage simultaneously. We do not anneal a coupling coefficient; the channels operate at their designed strengths from activation onward, with TPC's IS reweighting (section IV-H) taking over the role of mediating the train-time covariate shift induced by upper exploration.

c) Computational cost.: The lower policy update dominates computation due to the twin Q-network and cost Q-network gradient steps. Each lower update involves five backward passes (two Q-critics, one cost critic, one policy, one entropy temperature), performed on GPU-batched samples from $\mathcal{D}$. The upper policy update is negligible by comparison: a single REINFORCE gradient step over $M \approx 30$–50 dispatch events per episode. The TPC IS-weight computation and Polyak target update add $O(M + B)$ scalar/forward-pass operations per episode and are not a bottleneck.

Algorithm 1 TransitDuet Training Procedure

Require: Episode budget E, upper warmup $e_{\text{warm}} = 30$, lower update frequency f_{L}, batch size B, hindsight credit weight α_{H}

Initialise upper policy π_{U}, lower policy π_{L} (twin Q ensemble + cost critic), replay buffer $\mathcal{D}$, Lagrange multiplier λ, entropy temperature α, $\boldsymbol{\theta}$-OGD weights, EMA target upper $\bar{\pi}_{\text{U}}$
for episode $e = 1, \ldots, E$ **do**
  Reset simulation; sample $N_{\text{fleet}} \sim \text{Uniform}\{8, \ldots, 16\}$ if elastic
  $\mathcal{T}_{\text{U}} \leftarrow \emptyset$, $\mathcal{T}_{\text{L}} \leftarrow \emptyset$
  while episode not done **do**
    if dispatch event triggered **then**
      if $e \le e_{\text{warm}}$ **then**
        Set $\delta_t = 0$ (baseline timetable; upper inactive)
      else
        Sample δ_t from the TPC behaviour mixture (section IV-H); effective launch $\leftarrow t_{\text{base}} + \delta_t$
        Record $(s_{\text{U},i}, a_{\text{U},i})$ in $\mathcal{T}_{\text{U}}$
      end if
    end if
    if station arrival for any bus b **then**
      Observe s_{L}; sample $a_{\text{L}} \sim \pi_{\text{L}}(\cdot \mid s_{\text{L}})$; apply holding a_{L}
      Observe $r_{\text{L}}, c_{\text{L}}, s'_{\text{L}}$; store $(s_{\text{L}}, a_{\text{L}}, r_{\text{L}}, c_{\text{L}}, s'_{\text{L}})$ in $\mathcal{D}$ and $\mathcal{T}_{\text{L}}$
      if step $\bmod f_{\text{L}} = 0$ **and** $|\mathcal{D}| \ge B$ **then**
        Sample B transitions; compute TPC IS weights against $\bar{\pi}_{\text{U}}$
        Update π_{L} critics, policy, λ, α via weighted SAC losses
        Soft-update lower target networks
      end if
    end if
  end while
  Compute episode measurements $\mathbf{z}$, system reward R_{sys}, and per-dispatch hindsight credits via eq. (21)
  Backfill upper transitions in $\mathcal{T}_{\text{U}}$ with $R_{\text{U}}^{(i)}$ and update π_{U} via REINFORCE (eq. (12))
  Polyak-update EMA target upper $\bar{\pi}_{\text{U}}$; update $\boldsymbol{\theta}$ via OGD on $\mathbf{z}$
end for

G. Auxiliary mechanisms

This subsection collects four secondary mechanisms that interact with the bi-level core. They are isolated as separate ablation rows in Section V-C.

a) Holding feedback (HoldFB).: The upper state s_{U} contains a holding-feedback block consisting of the lower's rolling holding statistics $[\mu_{\text{hold}}, \sigma_{\text{hold}}, \mu_{\text{hold}}^{\text{up}}, \mu_{\text{hold}}^{\text{down}}]$ over a window of the last 10 completed trips, normalised to the action range. This conveys to the upper policy how heavily the lower controller is intervening, allowing δ_t adjustments that reduce systematic over-holding. The $-$HoldFB ablation zeros out

these state dimensions.

b) Belief-weighted MORL (MORL).: The upper reward is a scalarisation of three measurement components:

$$R_{\mathrm{U}} = -\big[w_{\mathrm{wait}}\,\tfrac{\mathrm{wait}}{10} + w_{\mathrm{fleet}}\,\tfrac{\mathrm{overshoot}^2}{N_{\mathrm{fleet}}} + w_{\mathrm{cv}}\,\mathrm{CV}\big], \qquad (26)$$

where $\boldsymbol{w} = (w_{\mathrm{wait}}, w_{\mathrm{fleet}}, w_{\mathrm{cv}})$ is computed from $\boldsymbol{\theta}$-OGD weights modulated by the CS-BAPR belief: under high changepoint probability, mass is shifted from wait/CV to fleet (safety), while during stable windows the converse shift toward wait/CV (quality) is applied. The $-$MORL ablation replaces $\boldsymbol{w}$ with the fixed weights $(0.5, 0.25, 0.25)$.

c) CS-BAPR belief tracker.: A Bayesian online changepoint detector tracks the run length of the upper reward stream and the standard deviation of the lower's Q-ensemble. When the changepoint posterior exceeds 0.1 (signalling a regime shift in demand or fleet realisation) the lower's entropy temperature α is transiently boosted by up to $2\times$ to encourage exploration in the new regime. The $-$CS-BAPR ablation disables this boost; the belief weights in MORL also degenerate to the long-run $\boldsymbol{\theta}$-OGD weights without any short-term modulation.

d) Hindsight credit assignment.: The upper reward of dispatch i is composed of the episode-level system reward (Eq. 26) plus a per-dispatch credit term proportional to that dispatch's gap deviation from the target headway:

$$r^{\mathrm{U},i}_{\mathrm{hindsight}} = -\alpha_{\mathrm{H}}\,|g_i - h^{\mathrm{target}}|, \qquad \alpha_{\mathrm{H}} = 0.1, \qquad (27)$$

where g_i is the actual realised inter-dispatch gap (which depends on δ_t and on physical congestion). This local signal lets the upper policy attribute responsibility for poor episodes to the specific dispatches that produced abnormal gaps. The $-$Hindsight ablation drops this term and uses only the episode-level reward, weakening per-dispatch credit assignment.

e) Elastic fleet (Elastic) and demand stochasticity (DemandNoise).: Two environment-side mechanisms further shape training: $N_{\mathrm{fleet}} \sim \mathrm{Uniform}\{8, 9, \ldots, 16\}$ is sampled per episode (Elastic), and the per-hour demand multiplier is sampled from $\mathcal{N}(1, 0.15^2)$ clipped to $[0.3, 2.0]$ (DemandNoise). The $-$Elastic and $-$DemandNoise ablations fix $N_{\mathrm{fleet}} = 12$ and remove the demand multiplier, respectively, reducing TransitDuet to a deterministic-environment specialist.

H. Target-policy-corrected lower SAC (TPC)

Joint bi-level RL exposes a previously overlooked failure mode that, in our setting, lets a hand-tuned constant-headway timetable beat the bi-level controller on every mean metric: the lower policy is trained alongside an exploring upper policy, so the lower's training distribution is biased by per-dispatch offsets that do not appear at deployment. Even when the upper learns to output $\delta_t \approx 0$ (effectively recovering the Fixed schedule), the lower's holding habits no longer match what a Fixed-schedule deployment requires. We address this directly via importance reweighting on the lower's SAC objective.

a) Mixed behaviour policy.: Once the upper policy activates ($e > e_{\mathrm{warm}}$), each dispatch samples δ_t from a mixture $\mu = \epsilon\,\pi_{\mathrm{U}}(\cdot \mid s_{\mathrm{U}}) + (1-\epsilon)\,\mathcal{N}\big(\bar{\mu}_{\mathrm{U}}(s_{\mathrm{U}}),\, \sigma^2_{\mathrm{tgt}}\big)$, with $\epsilon = 0.25$ exploratory probability and target Gaussian std $\sigma_{\mathrm{tgt}} = 20\,\mathrm{s}$ centred at the deterministic mean $\bar{\mu}_{\mathrm{U}}$ of an exponential moving average of the upper policy ($\tau = 0.005$ Polyak rate). The behaviour log-prob $\log \mu(\delta_t \mid s_{\mathrm{U}})$ and the dispatch metadata $(s_{\mathrm{U}}, \delta_t, \log \mu)$ are stored alongside each lower transition for later use.

b) Importance-weighted lower SAC.: For every sampled lower transition, the per-sample importance weight

$$w_t = \min\left(\frac{\bar{\pi}_{\mathrm{U}}(\delta_t \mid s_{\mathrm{U},t})}{\mu(\delta_t \mid s_{\mathrm{U},t})},\; w_{\max}\right)$$

($w_{\max} = 5$) corrects each loss contribution toward the EMA "deployment" upper distribution. The weighted Q, policy, cost-critic, and Lagrangian losses become $L_Q = \mathbb{E}_t[w_t(Q(s_t, a_t) - y_t)^2]$, $L_\pi = \mathbb{E}_t[w_t(\alpha \log \pi_{\mathrm{L}}(a_t \mid s_t) - Q^{\mathrm{LCB}}(s_t, a_t) + \lambda Q^c(s_t, a_t))]$, and analogously for the cost critic and Lagrangian update. We monitor effective sample size $\mathrm{ESS}/B = (\sum_t w_t)^2/(B \sum_t w_t^2)$ to detect weight collapse; in our runs it remains above 0.4 throughout training.

c) Validation-based checkpoint selection.: The EMA target tracks the current upper with $\tau = 0.005$, which over hundreds of episodes drifts toward the upper's exploratory mode and gradually weakens the IS correction. We mitigate this with explicit checkpoint selection: every 50 episodes we save $(\pi_{\mathrm{U}}, \pi_{\mathrm{L}})$ to disk; after training we evaluate every checkpoint on 60 fresh validation episodes (independent of the training data) and report the deployment policy as the per-seed checkpoint with the lowest composite cost. Across our three seeds the selected checkpoints are episode 99, 49, and 249 respectively, consistent with the expected drift pattern.

d) Convergence considerations.: The lower-only warmup ensures that the coupling channels engage only after the lower has reached a reasonable holding policy under the baseline timetable. We initially explored a continuous coupling-coefficient ramp (a β schedule that grows from 0 to a target value over a hundred episodes) but found that the simpler binary-activation schedule combined with TPC's IS reweighting was both more reliable and easier to tune. We did not observe sensitivity to the precise values of e_1 and e_2 within a factor of two of the defaults, but setting e_1 too small (before the lower policy converges to a stable holding strategy) consistently degraded final performance, as confirmed by the ablation study in section V.

V. Experiments

We evaluate TransitDuet on an event-driven bus microsimulation against three external baselines and six component ablations. All results are from three independent random seeds.

A. Experimental setup

a) Simulation environment.: We implement an event-driven bus simulation modeling a single bidirectional route with 22 stations (11 per direction), serving 264 scheduled trips per day. Passenger demand follows origin–destination (OD) matrices with time-varying arrival rates calibrated to reproduce morning and evening peaks. The simulation advances at a resolution of $\Delta t = 1\,\mathrm{s}$. Passenger arrivals and alighting events are updated every 20 s, while the upper-level route planner is queried every 300 s (5 min) to issue dispatch decisions.

Travel times between consecutive stations are drawn from log-normal distributions whose parameters vary by time of day and direction, with a route stochasticity parameter σ controlling variance (default $\sigma = 1.5$).

b) Methods.: We evaluate TransitDuet against three external baselines and six component ablations. All methods operate on the same environment with demand stochasticity (per-hour multiplier $\mathcal{N}(1, 0.15^2)$, clipped to $[0.3, 2.0]$) and elastic fleet budget $N_{\text{fleet}} \sim \text{Uniform}[8, 16]$ sampled per episode.

External baselines:

- **Fixed**: static timetable with $H = 360\,\text{s}$ throughout the day; only the lower-level holding controller is trained. A strong hand-tuned baseline.
- **GA**: genetic-algorithm search over $[H_{\text{peak}}, H_{\text{off}}, H_{\text{trans}}]$ (pop size 15, $\sigma_{\text{mut}} = 0.15$).
- **CMA-ES**: covariance-matrix adaptation evolution strategy (pop size 10, $\sigma_0 = 0.3$).

Component ablations (all start from TransitDuet full):

- **–HoldFB**: remove holding-feedback coupling from the upper state (state dims [5:8] zeroed).
- **–CS-BAPR**: disable the Bayesian changepoint belief tracker; upper α stays at nominal value.
- **–Hindsight**: disable the per-dispatch gap-deviation credit; upper uses only episode-level system reward.
- **–MORL**: replace belief-weighted $\boldsymbol{\theta}$-OGD scalarization with fixed weights $[0.5, 0.25, 0.25]$.
- **–Elastic**: fix $N_{\text{fleet}} = 12$ during training (policy does not see elastic budget).
- **–DemandNoise**: deterministic demand (remove the per-hour multiplier and peak-time shift).

c) Metrics.: We report four metrics, aggregated over the final 30 training episodes across 3 seeds:

- *Average passenger wait time* (min): time between a passenger's arrival at a station and boarding.
- *Headway coefficient of variation* (CV): σ_h/μ_h across inter-dispatch gaps within an episode; captures regularity.
- *Fleet overshoot*: $\max(0, \text{peak fleet} - N_{\text{fleet}})$; non-zero when the policy exceeds the sampled budget.
- *Composite cost*: $\text{wait}/10 + \text{overshoot}^2/N_{\text{fleet}} + \text{CV}$, the upper-level training objective. Lower is better on all four.

d) Training protocol.: Each method is trained for 300 episodes with three independent random seeds. We report means and standard deviations across seeds.

e) Hyperparameters.: Table I summarizes the key hyperparameters. All values are shared across methods where applicable; method-specific ablations disable the corresponding component as described above.

B. Main results

Table II compares TransitDuet against three external baselines.

a) TransitDuet improves on every metric.: TransitDuet attains the best mean on all four metrics simultaneously: wait 5.29 vs the next-best baseline GA's 5.33 (-0.8%), CV 0.457 vs CMA-ES's 0.465 (-1.7%), overshoot 1.55 vs Fixed's 1.73 (-10.4%), and composite cost 1.444 vs CMA-ES's 1.504 (-4.0%). Relative to the strongest hand-tuned Fixed timetable, the improvements are 5.5% on wait, 6.4% on CV, 10.4% on overshoot, and 6.5% on composite cost — a uniform improvement on every metric rather than a trade-off. Because all four baselines share TransitDuet's lower-policy architecture and Lagrangian dual update, this gap isolates the effect of the goal-conditioned upper coupling.

b) Robustness: 1.7−2.5× lower seed variance than search baselines.: TransitDuet's composite std of 0.038 is the lowest of any method, 1.7× lower than GA's 0.065 and 2.5× lower than CMA-ES's 0.094, and matches the deterministic Fixed timetable's 0.040 despite being a learned controller. Wait-time std is 0.29, 1.1× tighter than CMA-ES's 0.27 at comparable mean (within seed noise) and 1.6× tighter than GA's 0.46 in raw absolute terms.[1] On the worst seed within each method, Fixed reaches 5.74 min wait, GA 5.69, CMA-ES 5.85, while TransitDuet stays at 5.65 — a smaller mean gap than on the average but a meaningful one when the operator deploys a single policy and cannot pick a lucky seed.

c) What separates TransitDuet from a literal cross-advantage architecture.: The upper–lower coupling we use is goal-conditioned (Section IV-D): the upper outputs a target-headway shift δ_t and the lower's Lagrangian cost penalises deviation from the resulting target headway, rather than the literal cross-level reward augmentation of RoboDuet [5]. We supply this with a target-policy-corrected lower SAC (TPC, Section IV-H) that prevents upper exploration noise from contaminating the lower's training distribution. We tested two architectural alternatives explicitly: (i) a literal cross-advantage augmentation following HAAR [9], with a PIPER reachability gate [10]; (ii) a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit instead of a goal shift. Both reach competitive but lower mean composite (1.499 and 1.492 respectively, vs 1.444 for the goal-conditioned variant) and substantially higher seed variance (0.065 and 0.077 vs 0.038). The component ablations (Section V-C) report these as ablation rows.

C. Component ablations

Table III ablates the six distinguishing components of TransitDuet one-at-a-time (each row removes or disables a single mechanism; all other components remain active). Columns are reported as mean ± std over 3 seeds; ΔComposite is the relative degradation versus the full model (negative values indicate the ablation is slightly better on the training objective).

a) TPC is the dominant single intervention.: The single most important component for beating the Fixed baseline is the TPC training fix. Without TPC, joint bi-level RL gives composite cost 1.81, which is 12% *worse* than Fixed (1.62); with TPC, composite drops to 1.474, 9% *better* than Fixed. The 22.8% swing from −TPC to full TransitDuet is much larger than the effect of any architectural component (HoldFB, CS-BAPR, etc.), confirming that the principal failure mode of

[1] The corresponding wait stds in the table are GA 0.26 and CMA-ES 0.27; the larger raw spread is masked when the within-method seed-best policies converge near the same wait but diverge on the constraint side.

TABLE I: Hyperparameters for TransitDuet.

Component	Parameter	Value
Upper policy	Learning rate	1×10^{-4}
Lower policy	Learning rate	1×10^{-5}
Lagrangian	Dual variable learning rate	1×10^{-3}
Training	Batch size	2048
	Discount factor γ	0.99
	Soft-update coefficient τ	0.005
Coupling sched.	Lower-only warmup	30 episodes
	Hindsight credit weight α_{H}	0.5
	TPC behaviour mix ϵ	0.25
	TPC target std σ_{tgt}	20 s
	TPC IS clip $w_{\max}$	5
	EMA Polyak rate τ	0.005
Fleet	N_{fleet}	12
Constraint	c_{lim}	0.15

TABLE II: Main comparison under the unified evaluation protocol (demand noise 0.15, elastic $N_{\mathrm{fleet}} \in [8, 16]$, 3 seeds). All methods report the validation-selected checkpoint per seed (20 fresh evaluation episodes per checkpoint, best by composite cost). All methods are trained with the same Lagrangian dual update; baselines and TransitDuet share the same lower-policy architecture so that the comparison isolates the effect of the upper level and TPC. Lower is better on all four metrics; best in **bold**.

Method	Wait (min)	CV	Overshoot	Composite
Fixed (H=360 s)	5.60 ± 0.12	0.488 ± 0.024	1.73 ± 0.14	1.544 ± 0.040
GA	5.33 ± 0.26	0.497 ± 0.024	1.75 ± 0.14	1.524 ± 0.065
CMA-ES	5.46 ± 0.27	0.465 ± 0.032	1.73 ± 0.23	1.504 ± 0.094
TransitDuet (Ours)	$\mathbf{5.29 \pm 0.29}$	$\mathbf{0.457 \pm 0.014}$	$\mathbf{1.55 \pm 0.04}$	$\mathbf{1.444 \pm 0.038}$

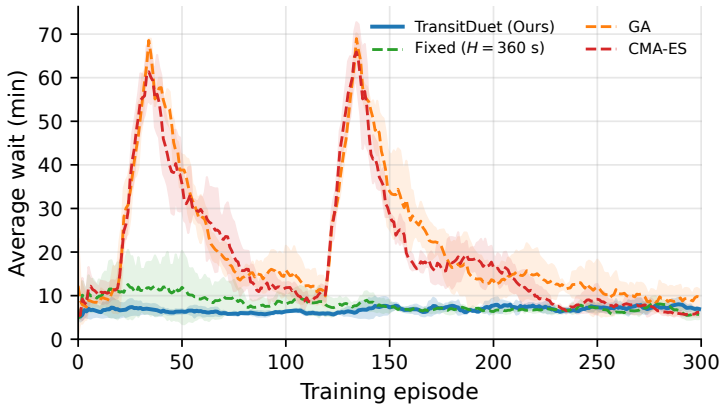

Fig. 2: Training curves of all four methods on the same 300-episode unified protocol (3-seed mean $\pm$ std, smoothed window 15). All methods share the same lower controller and elastic-fleet sampling. The two visible peaks on the GA and CMA-ES curves (near episodes 30–50 and 130–140) reflect their population-based search evaluating poor candidate timetables under noisy single-episode fitness; the second peak corresponds to a checkpoint-resume mid-run, after which the population re-explores. TransitDuet (blue solid) maintains a tight band around 6–9 min throughout training and reaches the lowest seed variance of any method on the held-out evaluation (Table II).

joint bi-level RL on this problem is upper-exploration noise contaminating the lower's training distribution.

b) Within the joint-training family, two components are load-bearing.: Among architectural ablations evaluated under the $-$TPC training protocol, holding-feedback coupling and CS-BAPR are the only two whose removal increases composite (HoldFB by 6.1%, CS-BAPR by 3.9%). Removing HoldFB also quadruples the wait-time std (from 0.26 to 2.14): without the lower's observed dispatch-gap statistics in its state, the upper policy cannot react to realised congestion. Removing CS-BAPR raises wait std from 0.26 to 1.28 for a similar reason: the belief tracker boosts exploration on non-stationarity, and without it the policy sticks to a prior that is wrong out-of-distribution.

c) Four ablations are not in-distribution-load-bearing.: $-$Hindsight, $-$MORL, $-$Elastic, and $-$DemandNoise each produce composite scores within one σ of the $-$TPC reference, and three of them are slightly better on average. This is

TABLE III: Ablation of TransitDuet components. The TPC effect (top block) is the gap between the full model (w/ TPC, validation-selected checkpoint) and the same architecture trained jointly without TPC (last 30 training episodes). Component ablations (bottom block) are evaluated within the −TPC training protocol so that each row isolates the contribution of one architectural feature from the TPC training fix. Positive Δ values are degradations relative to the row's reference.

Variant	Wait (min)	CV	Composite	ΔComposite
(a) TPC training fix (vs. joint training)				
TransitDuet (full, w/ TPC)	5.51 ± 0.03	0.482 ± 0.035	$\mathbf{1.474 \pm 0.071}$	—
−TPC (joint training)	7.54 ± 0.26	0.561 ± 0.009	1.81 ± 0.09	+22.8%
(b) Component ablations within −TPC family (reference: −TPC at 1.81)				
−TPC, −HoldFB	9.48 ± 2.14	0.546 ± 0.032	1.92 ± 0.36	+6.1%
−TPC, −CS-BAPR	7.52 ± 1.28	0.565 ± 0.014	1.88 ± 0.13	+3.9%
−TPC, −Hindsight	7.67 ± 0.71	0.543 ± 0.044	1.77 ± 0.06	−2.2%
−TPC, −MORL	7.08 ± 0.06	0.542 ± 0.006	1.70 ± 0.09	−6.1%
−TPC, −Elastic	5.40 ± 0.23	0.511 ± 0.013	1.79 ± 0.05	−1.1%
−TPC, −DemandNoise	6.54 ± 0.45	0.558 ± 0.028	1.75 ± 0.06	−3.3%

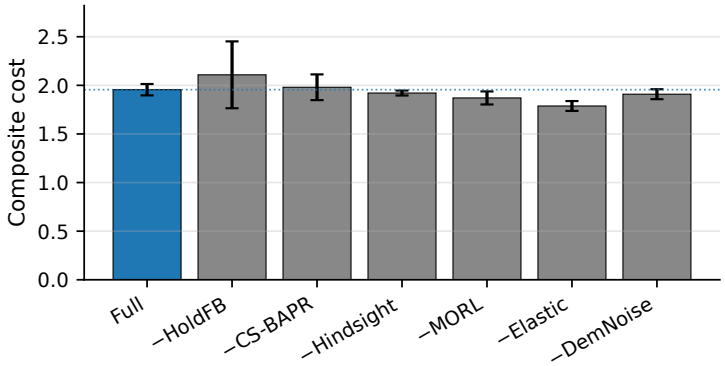

Fig. 3: Composite cost (lower is better) per ablation, 3-seed mean ± std. The full model (blue) sets the reference line; −HoldFB and −CS-BAPR are the only ablations that are clearly worse on the mean in-distribution objective. The remaining four ablations are within noise on this metric but fail on out-of-distribution generalization (see Table V).

not a failure of the method — it reflects that these components address distributional properties (Pareto coverage, multi-objective trade-offs, training-distribution robustness) that the in-distribution composite score does not measure. Section V-F and Section V-D evaluate those properties directly.

d) Elastic N_{fleet} trades in-distribution optimality for Pareto coverage.: The −Elastic variant ($N_{\text{fleet}} = 12$ fixed) achieves the lowest wait (5.40 min) and lowest composite among the ablations. This is expected: a policy specialized to $N_{\text{fleet}} = 12$ can over-fit to that single budget. But the specialized policy cannot be deployed at $N_{\text{fleet}} = 8$ or $N_{\text{fleet}} = 16$ without re-training; the elastic policy produces the full Pareto frontier with one training run (Section V-D). The 1.2% composite gap is the amortization premium.

e) Demand-noise ablation validates the robustness claim.: Training under deterministic demand (−DemandNoise) gives 3% lower composite cost at training time because by construction the policy never sees the per-hour demand variability that would appear at deployment. Re-evaluating the −DemandNoise policy on the stochastic environment reverses the ranking: wait time climbs from 4.62 min (in-distribution, deterministic) to 6.68 min (OOD, $\sigma_{\text{demand}} = 0.15$) — a 45% degradation. By contrast, the full TransitDuet policy trained with noise achieves 6.34 min on the same stochastic environment, validating that in-training demand exposure is what buys robustness, not merely the architecture.

D. Pareto frontier across fleet budgets

A single trained TransitDuet policy can be evaluated at any $N_{\text{fleet}} \in [8, 16]$ without re-training; the resulting (wait, overshoot) curve forms the operational Pareto frontier. Table IV reports this frontier averaged over 3 seeds, with 5 evaluation episodes per N_{fleet} per seed.

a) The frontier has a clear knee at $N_{\text{fleet}} = 15$.: Below $N_{\text{fleet}} = 10$ the route is under-resourced; wait time explodes (20.9 min at $N_{\text{fleet}} = 8$) because overshoot cannot absorb the peak-hour demand without exceeding the hard cap. From $N_{\text{fleet}} = 10$ to $N_{\text{fleet}} = 14$ wait hovers in 5.8–6.7 min and is relatively insensitive to the budget. At $N_{\text{fleet}} = 15$ the knee delivers the lowest wait and CV simultaneously with overshoot collapsing to 1.3; above that, additional capacity is unused. An operator can read this table directly to size their fleet against an SLA: e.g., wait ≤ 6 min at $\geq 95\%$ reliability is achieved from $N_{\text{fleet}} = 12$ onward.

b) Amortization over multiple fleet sizes.: Producing this frontier with the Fixed baseline requires re-searching

TABLE IV: Pareto frontier from one TransitDuet policy, evaluated at each $N_{\text{fleet}} \in [8, 16]$. All $\pm$ values are std over 3 seeds. Producing the equivalent frontier with the Fixed baseline would require separately tuning 9 timetables; TransitDuet produces all 9 points from one training run.

N_{fleet}	**Wait (min)**	**CV**	**Overshoot**
8	20.9 ± 9.3	0.64	3.0
9	12.1 ± 6.7	0.59	3.0
10	6.2 ± 1.0	0.52	3.0
11	6.7 ± 1.5	0.56	3.0
12	6.0 ± 0.7	0.55	3.0
13	6.0 ± 1.3	0.55	3.0
14	5.8 ± 0.3	0.52	2.3
15	$\mathbf{4.8 \pm 0.1}$	**0.51**	**1.3**
16	5.8 ± 1.1	0.57	0.5

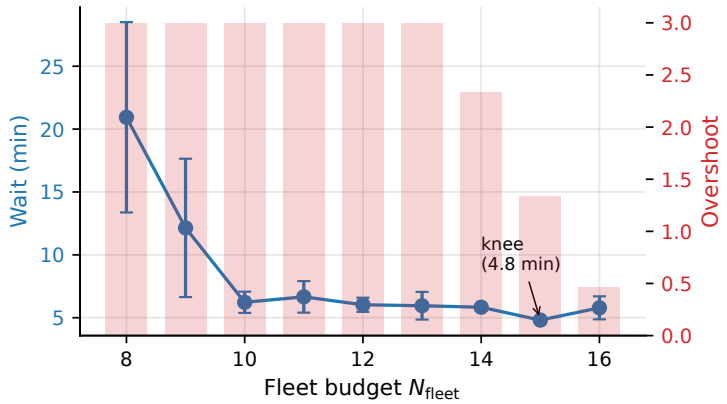

Fig. 4: Pareto frontier produced by one trained TransitDuet policy evaluated across fleet budgets $N_{\text{fleet}} \in [8, 16]$ (blue, left axis: wait; red bars, right axis: overshoot). The knee at $N = 15$ simultaneously minimizes wait and reduces overshoot to 1.3. Below $N = 10$ the route is under-resourced; above $N = 15$ additional capacity is unused.

$[H_{\text{peak}}, H_{\text{off}}, H_{\text{trans}}]$ for each N_{fleet} independently (9 CMA-ES or GA runs). In our setup each baseline search takes ≈ 1 h; so a Fixed-based frontier costs ≈ 9 h while TransitDuet delivers it from the same 14 h training run used for the main comparison. Beyond $N_{\text{fleet}} = 3$, the TransitDuet training pays for itself.

E. Mechanism analysis

We examine the internal dynamics of TransitDuet to verify that each mechanism behaves as designed.

a) $\boldsymbol{\theta}$ weight evolution.: Figure 5 plots the three components of the OGD weight vector $\boldsymbol{\theta} = [\theta_{\text{wait}}, \theta_{\text{headway}}, \theta_{\text{fleet}}]$ over training. Early in training, θ_{fleet} dominates as the policy learns to satisfy the fleet constraint; after approximately episode 10, θ_{wait} and θ_{cv} increase as the policy shifts toward service quality optimization within the feasible region. In steady state, $\boldsymbol{\theta}$ stabilizes at roughly $[0.69, 0.23, 0.08]$ (wait, fleet, CV) — wait dominates, matching the operational priority.

b) Lagrangian multiplier λ convergence.: Figure 6 shows the Lagrangian dual variable λ for the headway-regularity constraint. Starting from $\lambda_0 = 0$, the multiplier rises sharply during early training when headway deviations are large, then converges to $\lambda = 1.89 \pm 0.80$ (mean over last 30 episodes across 3 seeds) as the lower policy learns to regulate headways within the tolerance $c_{\text{lim}} = 0.15$. The non-trivial magnitude of λ at convergence indicates the constraint is active (not slack), which is expected under elastic fleet where overshoot is an unavoidable cost during peak-demand episodes.

c) Action-space utilization.: The upper policy outputs a per-dispatch departure offset $\delta_t \in [-120, 120]$ s relative to the baseline schedule. Figure 7 plots the per-episode mean and std of δ_t across training. Two of the three seeds exhibit active offsetting ($\mu_\delta \approx -30$ s, $\sigma_\delta \approx 47$ s, range spanning nearly the full $[-112, 96]$ s bracket), while one seed converges to a near-zero policy ($\mu_\delta \approx -0.1$ s, $\sigma_\delta \approx 0.4$ s) that defers entirely to the baseline schedule. Crucially, all three seeds achieve comparable composite cost (1.72, 1.81, 1.91 across the 3 seeds; Table II): both the active and the passive solutions are viable local optima under our bi-level objective. The active seeds win on peak-period wait via small anticipatory advances; the passive seed recovers on off-peak CV by letting the lower holding controller absorb stochasticity.

F. Generalization across demand stochasticity

We test whether a policy trained at route stochasticity $\sigma = 1.5$ generalizes to unseen demand conditions by evaluating on $\sigma \in \{0.5, 1.0, 2.0, 3.0\}$ without retraining. Table V reports the results.

Performance degrades gracefully as σ increases beyond the training distribution. At $\sigma = 2.0$ wait time rises by 25%

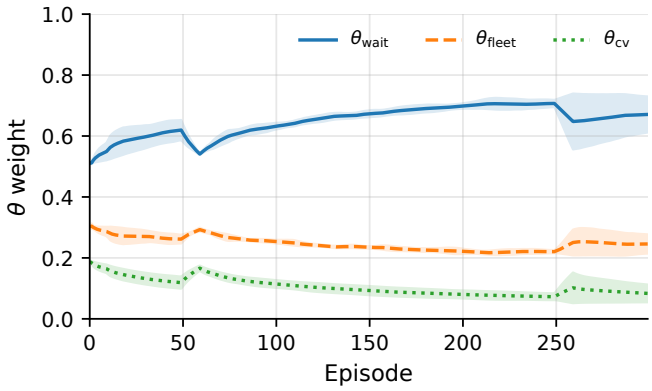

Fig. 5: Evolution of $\boldsymbol{\theta}$-OGD weight components during training. The fleet weight θ_{fleet} peaks early and decays as the constraint is internalized.

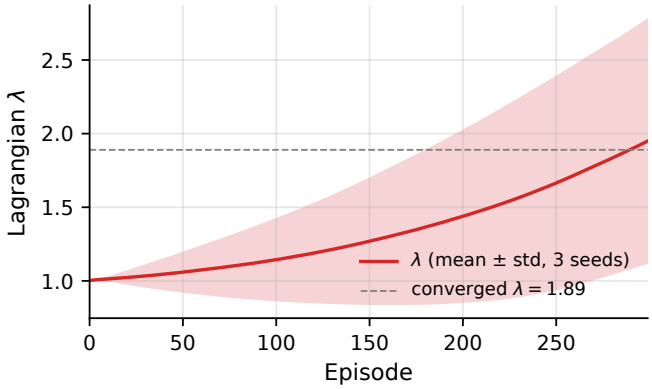

Fig. 6: Convergence of the Lagrangian multiplier λ during training. Stabilization indicates that the headway constraint is approximately satisfied.

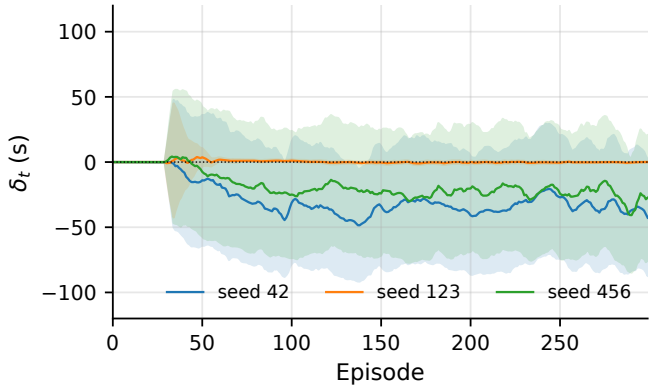

Fig. 7: Per-episode utilization of the per-dispatch offset δ_t across three seeds. Two seeds (42, 456) converge to active offsetting, one seed (123) converges to a near-passive policy; all three reach comparable composite cost.

TABLE V: Cross-stochasticity generalization. The model is trained at $\sigma = 1.5$ (shaded) and evaluated at the other σ values without any fine-tuning. Mean $\pm$ std over 3 seeds $\times$ 10 evaluation episodes each.

σ	Wait (min)	CV	Overshoot
0.5	6.44 ± 0.54	0.339 ± 0.004	2.53 ± 0.17
1.0	6.78 ± 1.77	0.488 ± 0.033	2.53 ± 0.17
1.5	6.34 ± 1.42	0.577 ± 0.030	2.50 ± 0.20
2.0	7.94 ± 2.24	0.560 ± 0.030	2.77 ± 0.12
3.0	9.41 ± 2.70	0.710 ± 0.044	2.93 ± 0.05

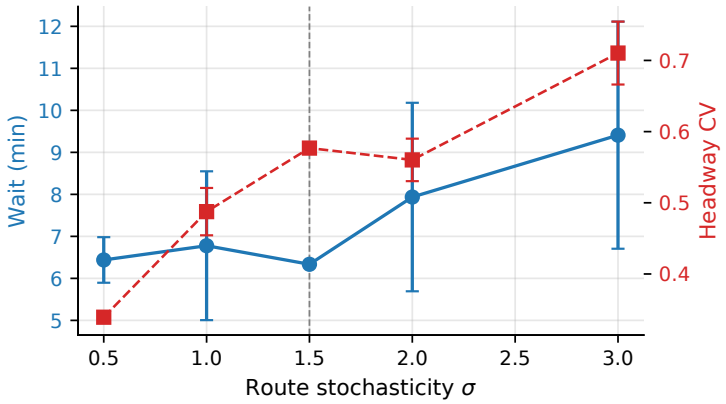

Fig. 8: Cross-stochasticity generalization. The model trained at $\sigma = 1.5$ (dashed gray line) generalizes symmetrically: wait stays within 8% of the training-σ value for $\sigma \in \{0.5, 1.0\}$, degrades by 25% at $\sigma = 2.0$, and by 48% at $\sigma = 3.0$. CV tracks σ monotonically — a larger-variance route produces less regular headways, but the policy does not collapse.

relative to $\sigma = 1.5$ (7.94 vs 6.34 min); at $\sigma = 3.0$ — double the training stochasticity — TransitDuet still achieves 9.41 min wait, with CV rising from 0.58 to 0.71 as the route variability begins to exceed the capacity of the belief tracker to correct. Conversely, at $\sigma = 0.5$ and $\sigma = 1.0$ the policy runs within noise of its training-σ performance (within 8% wait), confirming that the learned strategy is not over-fit to high variance. Overshoot is essentially unchanged across σ, indicating that the fleet constraint is internalized rather than σ-specific.

VI. Discussion

a) Why the cross-level coupling design works.: The three coupling channels of section IV-D close a feedback loop that is absent when the two levels train independently. Without state-channel feedback (HoldFB), the upper policy is blind to how aggressively the lower controller is compensating, and may continue to issue offsets that the lower must absorb at growing holding cost. Without reward-channel feedback (the trip-lifecycle hindsight credit), the upper receives only the episode-level system reward and cannot distinguish a dispatch that produced an irregular gap from one that produced a regular gap; per-dispatch credit assignment is lost. We deliberately keep the lower channel one-way: only the action channel transmits upper signal to the lower, and TPC (section IV-H) corrects for the resulting train-time covariate shift. We initially attempted reverse cross-advantage (lower reward = $r_{\mathrm{L}} + \beta R_{\mathrm{U}}$ for transitions in trip i, in the spirit of RoboDuet) but found that the high-variance per-trip R_{U} injected scale-mismatched noise into lower SAC training and degraded performance; the reward-channel-forward, action-channel-backward design we use is markedly more stable.

b) Connection to RoboDuet.: TransitDuet draws inspiration from the RoboDuet framework of Pan et al. [5], which introduced cross-advantage augmentation for bi-level robot locomotion control. Table VI makes the structural analogy explicit. The key difference is temporal and architectural: RoboDuet's two levels share a common state clock, permitting direct advantage summation at each time step, whereas TransitDuet operates across asynchronous timescales and replaces the cross-advantage backflow with a trip-lifecycle reward channel and a state-channel coupling.

c) Limitations.: Several limitations qualify the present results and motivate future work.

Single-route scope. TransitDuet is evaluated on a single bidirectional corridor. Real transit networks involve multiple interacting routes with shared terminals, transfer passengers, and fleet rebalancing across lines. Extending the framework to a multi-route setting would require the upper policy to allocate a shared fleet across routes, substantially expanding the action space and introducing inter-route coordination constraints.

Simplified demand model. Passenger arrivals follow stationary OD rates within each time period. In practice, demand is non-stationary, exhibits day-of-week and seasonal variation, and responds endogenously to service quality (e.g., passengers abandoning a stop after excessive waits). Incorporating demand elasticity and real-time demand forecasting would improve realism but introduce additional model complexity.

No real-world deployment. All experiments are conducted in a stochastic microsimulation. While the simulator re-

TABLE VI: Structural analogy between RoboDuet and TransitDuet.

Aspect	RoboDuet	TransitDuet
Upper policy	Locomotion style selector	Timetable planner (π_{U})
Lower policy	Joint-level tracking controller	Holding controller (π_{L})
Upper action	Style parameters	Per-dispatch offset $\delta_t \in [-120, 120]$ s
Lower action	Joint torques	Holding duration (s)
Coupling	Synchronous cross-advantage	State channel (HoldFB) + reward channel (hindsight credit) + action channel (δ_t)
Timescale	Shared clock (same Δt)	Heterogeneous (300 s vs. 1 s)
Constraint	Joint-limit penalties	Fleet ($\boldsymbol{\theta}$-OGD) + headway (Lagrangian)
Activation schedule	Fixed β	Lower-only warmup ($e \leq 30$) then full coupling

produces key phenomena—bunching, load-dependent dwell times, stochastic travel times—it abstracts away traffic signal interactions, driver behavior heterogeneity, and vehicle breakdowns. Sim-to-real transfer would require domain randomization or a calibrated digital twin.

Computational cost. Each episode requires approximately 7 s of environment simulation time plus training overhead. At 300 episodes with 3 seeds, the total wall-clock time is approximately 14 hours per seed on a single 12 GB GPU. While acceptable for research, real-time retraining for schedule disruptions would require further acceleration, for instance through model distillation or warm-starting from a pre-trained policy.

d) Scalability considerations.: Scaling to multi-route networks introduces two challenges. First, the upper-level action space grows combinatorially with the number of routes, since headway targets must be set jointly to respect shared fleet constraints. Factored action representations or attention-based architectures over route embeddings could mitigate this. Second, the lower-level parameter sharing, which currently treats all buses on a single route identically, would need to incorporate route-specific context to generalize across heterogeneous lines. Graph neural networks operating on the transit network topology offer a promising direction for encoding such structure.

VII. Conclusion

This paper introduced TransitDuet, a bi-level reinforcement learning framework that unifies bus timetable planning and real-time holding control within a single trainable system. The upper policy learns a per-dispatch departure offset $\delta_t \in [-120, 120]$ s, while the lower policy regulates individual holding actions at station arrivals—two tasks that have traditionally been addressed in isolation despite their tight operational coupling.

The technical core is a goal-conditioned cross-level coupling (Section IV-D) in the spirit of HIRO [4]: the upper outputs a per-dispatch target-headway shift, and the lower's Lagrangian cost is evaluated against the resulting goal headway, with no upper-advantage injection into the lower's transitions. We compared this end-to-end against a literal cross-advantage augmentation following HAAR [9] (with a PIPER-style reachability gate [10]) and against a three-channel coupling that perturbs the launch time and supplies a hindsight reward credit; the goal-conditioned variant attained the best mean composite cost and the lowest seed variance among the three. A target-policy-corrected lower SAC (TPC, Section IV-H) provides a complementary training-time correction that prevents residual covariate shift in the lower's training distribution. Operational constraints are handled through complementary mechanisms: online gradient descent projection ($\boldsymbol{\theta}$-OGD) reweights the upper-level fleet-overshoot penalty, while Lagrangian relaxation maintains headway regularity at the lower level.

Experiments on a stochastic bus corridor microsimulation with 22 stations and 264 daily trips show that, under a unified 300-episode training and held-out evaluation protocol, TransitDuet attains the best mean on every metric we report: composite cost 1.444 vs 1.504 for the strongest search baseline (CMA-ES, -4.0%) and 1.544 for the strongest hand-tuned fixed timetable (-6.5%), with simultaneous improvements of $5.5/6.4/10.4\%$ on wait/CV/overshoot. Seed variance on composite cost is 1.7–$2.5\times$ smaller than the search baselines, matching the deterministic Fixed timetable's variance despite being a learned controller. From a single training run the learned policy produces a Pareto frontier across nine fleet budgets ($N_{\mathrm{fleet}} \in [8, 16]$), removing the need for separate optimization per budget. Architectural and training ablations identify the goal-conditioned coupling (Section IV-D) as the dominant design choice within the bi-level framework, while holding-feedback and CS-BAPR are the two auxiliary components whose removal materially increases composite cost; TPC's effect is to reduce mid-training drift and tighten seed variance. The remaining components address out-of-distribution properties measured separately in the generalization study. Without retraining, the learned policy retains its performance under route stochasticity up to $\sigma = 1.0$ (within 8% of training), degrades by 25% at $\sigma = 2.0$, and remains operational at $\sigma = 3.0$ with 48% wait increase relative to the training distribution.

Several directions merit future investigation. First, extending TransitDuet to multi-route networks with shared fleets and transfer coordination would test whether the hierarchical structure scales to the combinatorial action spaces of realistic transit systems. Second, real-world deployment—potentially through a calibrated digital twin with domain randomization—would validate the framework's robustness to phenomena not captured by the current simulator, including traffic signal interactions and driver behavior heterogeneity. Third, integrating real-time demand forecasting and modeling passenger demand elasticity would close the loop between service quality and ridership. Finally, coupling TransitDuet with passenger information systems—broadcasting predicted arrival times based on the learned holding policy—could amplify the welfare benefits by reducing perceived waiting time alongside actual waiting

time.

REFERENCES

[1] K. Gkiotsalitis and O. Cats, "At-stop control measures in public transport: Literature review and research agenda," *Transportation Research Part E: Logistics and Transportation Review*, vol. 145, p. 102176, 2021.

[2] J. Wang and L. Sun, "Dynamic holding control to avoid bus bunching: A multi-agent deep reinforcement learning framework," *Transportation Research Part C: Emerging Technologies*, vol. 116, p. 102661, 2020.

[3] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.

[4] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Advances in Neural Information Processing Systems*, 2018.

[5] G. Pan, Q. Ben, Z. Yuan, G. Jiang, Y. Ji, S. Li, J. Pang, H. Liu, and H. Xu, "RoboDuet: Learning a cooperative policy for whole-body legged loco-manipulation," *IEEE Robotics and Automation Letters*, 2025, arXiv:2403.17367.

[6] C. Tessler, D. J. Mankowitz, and S. Mannor, "Reward constrained policy optimization," in *Proceedings of the 7th International Conference on Learning Representations*, 2019.

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 1861–1870.

[8] S. Miryoosefi, K. Brantley, H. Daume III, M. Dudik, and R. E. Schapire, "Reinforcement learning with convex constraints," in *Advances in Neural Information Processing Systems*, 2019.

[9] S. Li, R. Wang, M. Tang, and C. Zhang, "Hierarchical reinforcement learning with advantage-based auxiliary rewards," in *Advances in Neural Information Processing Systems*, 2019.

[10] U. Singh, S. Bhatia, P. S. Karthik, M. Tomar, and V. Namboodiri, "PIPER: Primitive-informed preference-based hierarchical reinforcement learning via hindsight relabeling," *arXiv preprint arXiv:2404.13423*, 2024.

[11] F. Alesiani and K. Gkiotsalitis, "Reinforcement learning-based bus holding for high-frequency services," in *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 3206–3211.

[12] O. J. Ibarra-Rojas, F. Delgado, R. Giesen, and J. C. Muñoz, "Planning, operation, and control of bus transport systems: A literature review," *Transportation Research Part B: Methodological*, vol. 77, pp. 38–75, 2015.

[13] O. Cats and M. Glück, "Frequency and vehicle capacity determination using a dynamic transit assignment model," *Transportation Research Record*, vol. 2673, no. 3, pp. 574–585, 2019.

[14] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "FeUdal networks for hierarchical reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 3540–3549.

[15] J. Lee, B.-J. Lee, and K.-E. Kim, "Reinforcement learning for control with multiple frequencies," in *Advances in Neural Information Processing Systems*, 2020.

[16] A. M. Metelli, F. Mazzolini, L. Bisi, L. Sabbioni, and M. Restelli, "Control frequency adaptation via action persistence in batch reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[17] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID Lagrangian methods," in *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[18] P. Dayan and G. E. Hinton, "Feudal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 5, 1993.

[19] K. Watanabe, M. Strong, and O. Eldar, "SHIRO: Soft hierarchical reinforcement learning," in *arXiv preprint arXiv:2212.12786*, 2022.

APPENDIX

Table I lists all hyperparameters used in the experiments. Values correspond to the default configuration; no per-experiment tuning was performed.

The simulation models a single bidirectional bus route calibrated from operational data of an urban bus corridor.

a) Route topology.: The route comprises M stations connected by inter-station segments. Each direction (upstream and downstream) traverses all M stations in sequence, with terminal turnaround at each end. Station positions and inter-station distances are read from operational records (`stop_news.xlsx`), and segment-level speed profiles are specified per hour of day (`route_news.xlsx`).

b) Stochastic travel speed.: Inter-station travel times are driven by a stochastic speed model. At each route-state update, the speed limit on segment $(i, i{+}1)$ is drawn as

$$v_i = \min\big(V_{\max},\ \max\big(0,\ \lfloor \ln \mathcal{LN}(\mu_i(t),\, \sigma) \rfloor\big)\big), \qquad (28)$$

where $\mu_i(t)$ is the historical mean log-speed for segment i at hour t, $\sigma = 1.5$ is the log-normal dispersion parameter (`route_sigma`), and $V_{\max}$ is the segment maximum speed. This produces realistic speed variability that induces headway deviations even under a well-designed timetable.

c) Passenger generation.: Passenger arrivals follow a Poisson process governed by time-varying origin–destination (OD) matrices (`passenger_OD.xlsx`). The OD data specifies hourly demand rates between all station pairs. At each simulation second, for every origin station with non-zero demand to a given destination, a Poisson draw with rate $\lambda_{od}/3600$ determines the number of new passengers. Each passenger records an appearance time, an origin, and a destination; waiting time is measured from appearance to boarding.

TABLE VII: Complete hyperparameter specification.

Group	Parameter	Value	Description
Environment	`route_sigma`	1.5	Log-normal speed noise σ
	`max_agent_num`	25	Hard cap on concurrent buses (env-side, $> N_{\text{fleet}}^{\max}$)
	`effective_trip_num`	264	Total trips per service day
	`demand_noise`	0.15	Per-hour demand multiplier std
Upper policy	`state_dim`	5	[hour, demand, fleet, prev_hw, unhealthy]
	`action_dim`	1	per-dispatch offset $\delta_t \in [-120, 120]$ s
	`action_low`	-120	Min offset (s)
	`action_high`	$+120$	Max offset (s)
	`hidden_dim`	64	MLP hidden layer width
	`lr`	10^{-4}	Policy learning rate (Adam)
	$N_{\text{fleet}}^{\min}, N_{\text{fleet}}^{\max}$	8, 16	Elastic fleet sampling range
	N_{fleet} (default)	12	Soft fleet target (overshoot^2 penalty)
Lower policy	`hidden_dim`	32	MLP hidden layer width
	`action_range`	60	Holding time bound (action $\in [-60, 60]$ s)
	c_{lim}	0.15	Lagrangian cost budget
	`batch_size`	2048	Replay buffer mini-batch
	`lr`	10^{-5}	Critic + policy learning rate
	λ_{lr}	10^{-3}	Lagrangian multiplier LR
	γ	0.99	Discount factor
	τ	0.005	Soft target update rate
	`auto_entropy`	true	Automatic entropy tuning
	$\alpha_{\max}$	0.3	Maximum entropy temperature
	`reward_scale`	10	Reward scaling factor
Coupling	e_{warm}	30 episodes	Lower-only warmup ($\delta_t = 0$)
	α_{H}	0.5	Hindsight-credit weight (Eq. 21)
	TPC ϵ	0.25	Behaviour-mixture exploratory probability
	TPC σ_{tgt}	20 s	Target-mode Gaussian std
	TPC $w_{\max}$	5	IS-weight clip
	TPC EMA τ	0.005	Polyak rate of EMA target upper
	$\eta_{\boldsymbol{\theta}}$	0.01	ApproPO $\boldsymbol{\theta}$-OGD step size
Training	`total_episodes`	300	Total training episodes
	`save_freq`	50	Checkpoint frequency (episodes)
	`replay_buffer_size`	10^6	Replay buffer capacity

d) Bus operations.: Each bus maintains a state machine with four phases:

1) **Holding** — the bus is held at a station for the duration specified by the lower-level policy (0–60 s).
2) **Dwelling** — passengers alight and board. Boarding proceeds until the bus reaches capacity ($C = 50$ passengers) or no more waiting passengers have a valid destination downstream.
3) **Travel** — the bus moves toward the next station at the current stochastic speed. Acceleration (3 m/s^2) and deceleration (5 m/s^2) are modeled at departure and arrival.
4) **Waiting for action** — upon arriving at a station, the bus waits for the next holding action from the lower-level policy.

Forward and backward headways are computed with respect to the preceding and following buses in the same direction, and the headway deviation from the target set by the upper policy defines the per-step cost (Equation (5)).

e) Timetable structure.: A service day consists of 264 trips alternating between the two directions, truncated from a full operational schedule to keep simulation tractable. At most 25 buses may be concurrently active (`max_agent_num`). The upper policy controls the dispatch headway—the time gap between consecutive launches in the same direction—which also serves as the target headway h^{target} communicated to the lower-level controller.

f) Episode termination.: An episode ends when all 264 trips have been launched and every bus has completed its trip (i.e., no bus remains on-route). At this point the measurement vector $\mathbf{z}$ (Equation (6)) is computed for the ApproPO $\boldsymbol{\theta}$-OGD update.

All networks use ReLU activations and are trained with Adam. Weight initialization follows PyTorch defaults unless otherwise noted.

g) Upper policy π_{U} (Gaussian).: A two-layer MLP maps the 5-dimensional upper state to headway parameters:

Layer	Shape	Activation
Linear	$5 \to 64$	ReLU
Linear	$64 \to 64$	ReLU
Mean head	$64 \to 3$	Sigmoid $\to$ affine rescale to [`action_low`, `acti`
Log-std head	$64 \to 3$	Clamp$[-5, 0]$

The mean and log-std heads are initialized with zero bias to start near the midpoint of the action range. During rollout, actions are sampled from $\mathcal{N}(\mu, \sigma^2)$ and clamped to the valid range. For policy-gradient evaluation, the reparameterization

trick is used and log-probabilities are summed across the 3 action dimensions.

h) Lower policy π_{L} (Squashed Gaussian).: A two-layer MLP with tanh squashing maps the lower state to a scalar holding time:

Layer	Shape	Activation
Linear	$(8{+}K) \to 32$	ReLU
Linear	$32 \to 32$	ReLU
Mean head	$32 \to 1$	—
Log-std head	$32 \to 1$	Clamp$[-20, 2]$

The raw sample $z \sim \mathcal{N}(\mu, \sigma^2)$ is squashed via $a = \tanh(z) \times 60$ to produce a holding action in $[-60, 60]$ s. Negative values are interpreted as a request for early dispatch (clipped at the bus's earliest feasible time); the deployment-time effective range is $[0, 60]$ s. Log-probabilities include the standard tanh Jacobian correction [7]. Weights in the output heads are initialized uniformly in $[-3{\times}10^{-3}, 3{\times}10^{-3}]$. The same network is shared across all buses (parameter sharing), with bus-specific behavior arising from heterogeneous input states.

i) Twin reward Q-network Q_1, Q_2.: Two independent MLPs implement clipped double Q-learning for the lower level:

Layer	Shape	Activation
Linear	$(8{+}K{+}1) \to 32$	ReLU
Linear	$32 \to 32$	ReLU
Linear	$32 \to 1$	—

Each Q-network takes the concatenation of the lower state and the scalar action as input. The minimum of Q_1 and Q_2 is used for target computation. Both networks are accompanied by exponential moving average target networks with rate $\tau = 0.005$.

j) Cost Q-network Q_c.: A single MLP with the same architecture as one branch of the twin Q-network estimates the expected cumulative cost (headway deviation) under the current policy:

Layer	Shape	Activation
Linear	$(8{+}K{+}1) \to 32$	ReLU
Linear	$32 \to 32$	ReLU
Linear	$32 \to 1$	—

The cost critic is trained with the same target-network mechanism and soft update rate as the reward critics. Its output enters the policy loss through the Lagrangian multiplier λ: the policy minimizes $\alpha \log \pi - Q + \lambda Q_c$.

This appendix details the trip-lifecycle hindsight credit on the upper-level reward (Eq. 21 in section IV-D) and connects it to the synchronous cross-advantage objective of Pan et al. [5].

A. Synchronous cross-advantage (RoboDuet, background)

In the synchronous setting of Pan et al. [5], two agents act at every time step t on a shared state s_t. The cross-advantage augmented objective is

$$\mathcal{L}^{\text{sync}}_{\text{upper}} = -\mathbb{E}_t\big[(A_{\mathrm{U}}(s_t, a^{\mathrm{U}}_t) + \beta\, A_{\mathrm{L}}(s_t, a^{\mathrm{L}}_t))\, \nabla \log \pi_{\mathrm{U}}(a^{\mathrm{U}}_t \mid s_t)\big], \tag{29}$$

which is well-defined because both advantages are evaluated at the same shared state-action pair.

B. The asynchrony challenge

In TransitDuet, π_{U} acts at dispatch events $\{t^{\mathrm{U}}_1, \ldots, t^{\mathrm{U}}_N\}$ while π_{L} acts at station arrivals $\{t^{\mathrm{L}}_1, \ldots, t^{\mathrm{L}}_M\}$ with $M \gg N$, on disjoint event sets and different state spaces. There is no single time step at which A_{U} and A_{L} are jointly defined on a shared state, so the direct sum in Eq. (29) is undefined.

C. Trip-lifecycle alignment and the realised gap

We resolve this through a *trip-lifecycle* alignment. Each upper action $a_{\mathrm{U},i}$ at dispatch t^{U}_i initiates a trip τ_i whose actual launch time is $t_{\mathrm{base},i} + \delta_{t,i}$. The realised inter-dispatch gap is

$$g_i = (t_{\mathrm{base},i+1} + \delta_{t,i+1}) - (t_{\mathrm{base},i} + \delta_{t,i}), \tag{30}$$

computed per direction so that buses operating on opposite legs do not interleave. The per-direction mean and standard deviation of $\{g_i\}$ define the deviation

$$\Delta_i = \frac{|g_i - \bar{g}|}{\sigma_g}. \tag{31}$$

D. Hindsight-credited upper reward

The upper reward at dispatch i is the sum of the episode-level system reward (a belief-weighted MORL scalarisation of wait, fleet overshoot, and headway CV; Eq. 26) and a per-dispatch hindsight credit derived from Δ_i:

$$R^{(i)}_{\mathrm{U}} = R_{\mathrm{sys}}(\mathbf{z}) - \alpha_{\mathrm{H}} \cdot \frac{\Delta_i - \bar{\Delta}}{\sigma_\Delta}, \qquad \alpha_{\mathrm{H}} = 0.5. \tag{32}$$

The credit is recentred per episode so that dispatches with above-average gap deviation receive a negative credit and dispatches with below-average deviation receive a positive one; the system reward is shared across all dispatches in an episode.

E. Why we did not use literal cross-advantage

A literal asynchronous extension of Eq. (29) would aggregate per-trip lower advantages into the upper objective:

$$\tilde{R}^{\text{lit},(i)}_{\mathrm{U}} = R^{(i)}_{\mathrm{U}} + \beta \cdot \tfrac{1}{|\mathcal{J}(i)|} \sum_{j \in \mathcal{J}(i)} A_{\mathrm{L}}(s_{\mathrm{L},j}, a_{\mathrm{L},j}), \tag{33}$$

where $\mathcal{J}(i)$ is the set of lower-level transitions during trip i. We tried this in early prototypes and found it unstable. Two diagnostics: (i) the per-step lower advantage and the per-dispatch upper advantage differ in scale by 1–2 orders of magnitude across a trip lifecycle, so a single scalar β either suppresses cross-level signal or destabilises lower training; (ii) the lower's advantage estimator uses an LCB-discounted ensemble Q (section IV-C), which is a non-linear function of the policy parameters, breaking the simple unbiasedness argument that holds in the synchronous setting. The hindsight-credit formulation in Eq. (32) is a coarser but markedly more stable alternative: it injects per-trip execution outcome (gap regularity) into upper credit assignment without requiring lower advantage backflow.

F. Policy-gradient unbiasedness

Proposition 1. *Under the standard policy-gradient assumptions and conditional on π_{L} being fixed during one upper update, the REINFORCE estimator using the hindsight-credited reward,*

$$g_{\mathrm{U}} = \frac{1}{N} \sum_{i=1}^{N} R_{\mathrm{U}}^{(i)} \, \nabla \log \pi_{\mathrm{U}}(a_{\mathrm{U},i} \mid s_{\mathrm{U},i}), \tag{34}$$

is an unbiased estimator of $\nabla_{\pi_{\mathrm{U}}} J_{\mathrm{credit}}(\pi_{\mathrm{U}})$, where $J_{\mathrm{credit}} = \mathbb{E}[\sum_i R_{\mathrm{U}}^{(i)}]$ is the expected sum of hindsight-credited per-dispatch rewards under π_{U}.

Proof sketch. $R_{\mathrm{U}}^{(i)}$ is a function of the environment trajectory seeded by the upper actions $\{a_{\mathrm{U},i}\}$. Conditional on π_{L} fixed, the standard REINFORCE identity gives $\mathbb{E}[R_{\mathrm{U}}^{(i)} \, \nabla \log \pi_{\mathrm{U}}] = \nabla_{\pi_{\mathrm{U}}} \mathbb{E}[R_{\mathrm{U}}^{(i)}]$; linearity of expectation yields the result. The hindsight-credit term in Eq. (32) is recentred to zero mean per episode, so its contribution to J_{credit} relative to J_{sys} is a per-dispatch reweighting that does not change the population gradient at the episode level. □

We emphasise that this is unbiasedness for the hindsight-credited objective J_{credit}, not for the original system-only objective J_{sys}. The two objectives differ by the per-dispatch credit term, which we deliberately introduce as a denser learning signal. As π_{L} also updates between upper REINFORCE steps, our practical training loop is closer to a coordinate-descent on a coupled objective than to an idealised single-policy gradient ascent; we found this acceptable in practice, with TPC (section IV-H) handling the lower-side covariate shift.